EVALUATION OF SCHEDULING EFFECTS OFF TinyML INFERENCE ON ESP32 USING FreeRTOS

by

Anh Le

A project submitted in partial fulfillment of the

requirements for the degree of Master of Science in Computer Science

Department of Computer Science and Cybersecurity

College of Sciences

Metropolitan State University

St. Paul, Minnesota

May 10, 2025

Disclaimer

I declare that I am the sole author of this thesis and that I have used no sources other than those listed in the references or cited in the text. I further declare that this work has not been submitted to any other institution for the purpose of obtaining a degree.

Abstract

The deployment of Tiny Machine Learning (TinyML) on resource-constrained embedded platforms enables low-latency, on-device intelligence for Internet of Things (IoT) applications, but it also introduces important timing and schedulability challenges. This thesis presents the design, implementation, and experimental evaluation of a real-time embedded inference system on the ESP32-S3 using TensorFlow Lite Micro (TFLM) and the FreeRTOS operating system. The system executes a quantized human activity recognition model while sharing processor time with concurrent sensing, communication, and I/O tasks that emulate realistic multitasking workloads found in embedded IoT environments.

To study the timing behavior of on-device inference under contention, a task-based software architecture is developed with fine-grained instrumentation for measuring inference latency, execution-time variability, observed worst-case execution time (WCET), and deadline miss ratio. The evaluation includes baseline execution, periodic workload interference, and multiple priority configurations that range from favorable scheduling conditions to heavily contended and starvation-prone scenarios. In addition to timing analysis, the thesis examines runtime resource usage relevant to embedded deployment, including memory footprint and workload-induced execution variability.

The experimental results show that inference performance is strongly shaped by scheduler configuration and competing real-time activities. Under favorable priority assignment, the inference task remains near the desired real-time bound with relatively low variability and a low miss rate. Under contended scheduling, latency and jitter increase substantially, and deadline misses become frequent. In severe low-priority conditions, the inference task may experience prolonged blocking or starvation, demonstrating that model deployment alone is insufficient to guarantee real-time behavior. These findings confirm that scheduling policy, task interference, and system-level design decisions are critical to the dependable use of TinyML in embedded systems.

This thesis contributes a practical evaluation framework for studying real-time TinyML behavior on microcontroller-based platforms and provides evidence that scheduling-aware system design is essential for achieving reliable and predictable edge intelligence.

Keywords: TinyML, real-time systems, FreeRTOS, embedded AI, scheduling, ESP32, latency, WCET)

Dedication

I would like to sincerely thank my advisor, Professor Sena Houeto, for her guidance, encouragement, and patience throughout this thesis. Her support helped me refine my ideas and continue moving forward even when the work became challenging.

I would also like to thank my friends, who have granted me boundless strength and courage through their kindness, companionship, and encouragement. Their presence made this journey lighter, and their support gave me the motivation to keep going.

Acknowledgements

It is appropriate to acknowledge financial support and faculty/students who assisted in your research. [cite: 19]

Contents

1 Introduction **1**

1.1 Motivation 1

1.2 Problem Statement 2

1.3 Research Gap 3

1.4 Research Objectives 4

1.5 Approach Overview 5

1.6 Contributions 6

1.7 Thesis Organization 6

2 BACKGROUND **8**

2.1 Embedded AI and TinyML 8

2.1.1 Embedded Inference on Microcontrollers 9

2.1.2 Resource Constraints in TinyML Systems 10

2.1.3 TinyML Deployment Pipeline 10

2.2 Real-Time Embedded Systems 11

2.2.1 Hard and Soft Real-Time Systems 11

2.2.2 Deadlines and Timing Constraints 12

2.2.3 Predictability and Determinism 12

2.2.4 Implications for Embedded AI Systems 13

2.3 FreeRTOS Task Model and Scheduling 13
2.3.1 Task Abstraction 13
2.3.2 Task States 14
2.3.3 Priority-Based Preemptive Scheduling 14
2.3.4 CPU Time Allocation and Task Interaction 15
2.3.5 Implications for Multitasking Systems 15
2.4 Scheduling Challenges in Multitasking Systems 16
2.4.1 Resource Contention 16
2.4.2 Starvation 16
2.4.3 Priority Imbalance 17
2.4.4 Non-Deterministic Execution 17
2.4.5 Impact on Timing Behavior 18
2.4.6 Relevance to Embedded AI Workloads 18
2.5 Timing Metrics in Real-Time Systems 18
2.5.1 Latency 19
2.5.2 Jitter 19
2.5.3 Worst-Case Execution Time (WCET) 19
2.5.4 Deadline Miss Ratio 20
2.5.5 Lateness 20
2.5.6 Summary and Relevance 20
2.6 TinyML in Embedded IoT Systems 21
2.6.1 Inference as a System-Level Task 21
2.6.2 Interaction with Communication and I/O Workloads 22
2.6.3 Implications for Real-Time Performance 22
2.6.4 Bridge to This Work 23

3 Related Works **24**

3.1 Real-Time Scheduling in Embedded Systems 24

3.2 Embedded AI Applications in IoT 26
3.3 Real-Time Scheduling in Embedded Systems 27
3.4 Timing Analysis and WCET Estimation 29
3.5 Summary and Research Gap 31

4 System Design **33**
4.1 Design Goals and Requirements 33
4.2 Target Platform and Software Environment 34
4.2.1 Hardware Platform 34
4.2.2 Software Stack 35
4.3 Overall System Architecture 37
4.4 TinyML Model and Inference Pipeline 38
4.5 FreeRTOS Task Architecture 40
4.6 Scheduling and Interference Design 41
4.7 Background Workload Design 43
4.8 Timing Instrumentation and Logging Mechanism 45
4.9 Memory and Resource Considerations 47
4.10 Chapter Summary 49

5 Experimental Setup **50**
5.1 Experimental Objectives 50
5.2 Testbed and Execution Environment 51
5.2.1 Hardware Platform 51
5.2.2 Software Stack 51
5.2.3 Core Allocation and Execution Constraints 51
5.2.4 Logging and Storage Environment 52
5.3 Deployed Model and Inference Configuration 52
5.4 Background Workload Design 53

5.4.1 Sensor Task Design . . . 54
5.4.2 Communication Task Design . . . 54
5.4.3 I/O Task Design . . . 55
5.4.4 Rationale for Controlled Interference . . . 55
5.5 Experimental Variables and Controlled Factors . . . 56
5.5.1 Independent Variables . . . 56
5.5.2 Dependent Variables . . . 57
5.5.3 Controlled Factors . . . 57
5.6 Task Parameters Used in the Experiments . . . 58
5.7 Evaluated Scheduling Scenarios . . . 58
5.7.1 Baseline Scenario . . . 59
5.7.2 Periodic Interference Scenario . . . 59
5.7.3 Priority-A Scenario . . . 59
5.7.4 Priority-B Scenario . . . 60
5.7.5 Priority-C Scenario . . . 60
5.8 Task Priorities for Evaluated Scheduling Scenarios . . . 61
5.9 Measurement and Logging Procedure . . . 61
5.10 Data Processing and Analysis Workflow . . . 63
5.11 Validity Considerations and Limitations . . . 64
5.12 Chapter Summary . . . 65

6 Results and Analysis **66**

6.1 Priority-A Scheduling Configuration . . . 67
6.2 Priority-B Scheduling Configuration . . . 68
6.3 Comparative Timing Analysis . . . 69
6.4 Deadline Miss and Lateness Analysis . . . 72
6.5 Priority-C Starvation Behavior . . . 73
6.6 Memory Usage . . . 75

6.7 Chapter Summary 76

7 Discussion **77**

7.1 Interpretation of Scheduling Effects 77

7.2 Model Efficiency Is Necessary but Not Sufficient 79

7.3 Scheduling-Aware Mitigation Guidelines 80

7.3.1 Measure Isolated Inference Latency Before Integration 81

7.3.2 Assign Inference Priority According to Timing Criticality 81

7.3.3 Bound Background Workload Duration and Frequency 82

7.3.4 Keep Logging Outside the Inference Critical Path 82

7.3.5 Treat Starvation as a Schedulability Failure 83

7.4 Optional Mitigation-Oriented Priority Configuration 84

7.5 Energy Implications 85

7.6 Generalizability of Findings 86

7.7 Limitations and Threats to Validity 87

7.7.1 Single Hardware Platform 87

7.7.2 Single Model and Application Type 87

7.7.3 Controlled Background Workloads 88

7.7.4 Observed WCET Rather Than Formal WCET 88

7.7.5 Fixed Deadline Configuration 88

7.7.6 No Direct Energy Measurement 89

7.8 Implications for Real-Time TinyML Design 89

7.9 Chapter Summary 90

8 Conclusion **91**

References Cited **94**

A Important Stuff **96**

List of Tables

4.1 Software Stack of the Proposed System 36
4.2 Task Roles in the Proposed System 42

5.1 Experimental Variables and Controlled Factors 57
5.2 Task Parameters Used in the Experiments 58
5.3 Task Priorities for Evaluated Scheduling Scenarios 61

6.1 Timing Results for Priority-A Configuration 67
6.2 Timing Results for Priority-B Configuration 68
6.3 Comparison of Priority-A and Priority-B Timing Results 70
6.4 Observed Starvation Behavior under Priority-C 74
6.5 Memory Usage of TinyML Inference System 75

List of Figures

4.1 System Architecture of the Embedded TinyML Inference Framework. The ESP32-S3 executes the inference task under FreeRTOS scheduling alongside sensor, communication, and I/O workload tasks, with timing instrumentation and logging support for runtime performance evaluation. 38

6.1 Empirical cumulative distribution of inference latency under Priority-A and Priority-B scheduling. Priority-A remains tightly concentrated near the nominal execution range, whereas Priority-B shows substantially greater dispersion and heavier tail behavior under increased contention. 71

Chapter 1

Introduction

Intelligent processing is becoming an increasingly important capability in modern Internet of Things and embedded systems. As machine learning functions move closer to the edge, resource-constrained devices are expected not only to perform inference locally but also to respond in a timely and dependable manner. This shift creates new opportunities for low-latency and autonomous operation, especially in applications where delayed decisions can reduce system effectiveness. However, the successful deployment of embedded inference depends on more than model accuracy or memory efficiency alone. It also requires understanding how inference behaves when it executes alongside other concurrent activities in a real-time multitasking environment. This thesis, therefore, investigates the timing behavior of embedded TinyML inference under realistic system-level scheduling and workload conditions.

1.1 Motivation

The rapid growth of the Internet of Things (IoT) has increased the demand for intelligent processing directly at the network edge [1, 2, 3]. In many embedded applications, devices are expected not only to sense and transmit data but also to interpret local conditions and respond quickly without relying on constant connectivity to external systems. This shift is especially important in domains such as health monitoring, industrial sensing, and safety-critical systems, where delayed decisions can reduce effectiveness or compromise reliability [1, 2, 3].

Traditional cloud-based approaches are not always well-suited to these environments. Sending raw sensor data to a remote server for inference introduces communication latency, increases bandwidth usage, and creates a dependence on network availability [4, 5, 6]. In addition, transmitting sensitive or continuous data streams may raise privacy and security concerns. For embedded systems operating in time-sensitive or connectivity-limited settings, these limitations make local

inference an increasingly attractive alternative [4, 1].

Tiny Machine Learning (TinyML) has emerged as a practical way to bring machine learning capabilities to resource-constrained embedded platforms [7, 8]. By enabling compact models to run on low-cost microcontrollers, TinyML supports fast local decision-making while reducing dependence on cloud infrastructure. This makes it possible to deploy intelligent behavior in small, low-power devices that must operate autonomously in real time [7, 8, 1].

However, the ability to execute inference on embedded hardware does not by itself guarantee dependable system behavior. In real deployments, inference rarely runs in isolation. It must share processor time and memory resources with sensing, communication, logging, and input/output activities, often under a real-time operating system [9, 10]. Under these conditions, inference latency may increase, timing variation may become significant, and deadlines may be missed even when the model itself appears computationally lightweight [10, 11, 12].

This creates an important gap between model-level feasibility and system-level predictability. A model that performs well in standalone evaluation may behave very differently once integrated into a multitasking embedded environment. For real-time TinyML applications, understanding this gap is essential because the usefulness of local intelligence depends not only on accuracy but also on whether inference can be completed within required timing constraints [7, 9, 10]. This thesis is motivated by the need to evaluate how embedded TinyML inference behaves under realistic workload interference and scheduling conditions to better understand its timing reliability in real-time systems.

1.2 Problem Statement

Although TinyML enables machine learning inference on resource-constrained embedded platforms, real-world deployment introduces challenges that are not visible in standalone model evaluation [7, 8, 1]. In practical embedded systems, inference does not execute in isolation. It must share limited processor time with other recurring activities such as sensor acquisition, communication, data logging, and input/output handling. These functions are often managed under

a real-time operating system using a multitasking scheduler, where task priorities and release patterns determine how processor time is allocated [9, 10].

In systems based on FreeRTOS-style multitasking, the timing of inference depends not only on its own computational cost but also on its interaction with concurrent tasks. Preemption, priority assignment, and resource contention can all affect when inference begins and how long it takes to complete [9, 10, 11, 12]. As a result, a model that appears lightweight in terms of memory usage or average execution time may still experience significant timing degradation once integrated into a running system.

This creates a fundamental system-level problem for real-time embedded TinyML. Under scheduling interference, inference latency may increase beyond its expected standalone duration, execution time may vary from one activation to another, and periodic jobs may miss their deadlines. These effects appear as latency inflation, jitter, and deadline misses, all of which reduce predictability and may compromise the reliability of time-sensitive applications [10, 11, 12].

The problem addressed in this thesis is therefore the gap between model-level efficiency and real-time system behavior. A machine learning model may be compact and computationally efficient, yet still fail to meet timing requirements when executed alongside competing workloads in a multitasking embedded environment. **Model-level efficiency does not guarantee real-time correctness.**

1.3 Research Gap

Current research on TinyML and real-time embedded systems has progressed along largely separate directions. Warden and Situnayake [8], David et al. [7], and Zhang and Li [1] emphasized the feasibility of deploying machine learning on resource-constrained embedded platforms, with particular attention to compact models, framework support, model compression, inference efficiency, and low-power operation. Similarly, Amirtharaj [13] highlighted the importance of energy profiling and efficiency in IoT-oriented embedded devices. However, most of this work evaluates embedded intelligence primarily in isolation and gives limited attention to the timing behavior of inference

when it is executed alongside other concurrent real-time activities.

By contrast, research in real-time systems has long focused on scheduling, blocking, synchronization, and timing predictability in multitasking environments. Sha et al. [11] and Davari and Sha [12] examined the effects of priority inversion and blocking on real-time schedulability, while Miranda et al. [10] analyzed FreeRTOS overheads and their effect on periodic task timing. Lin and Wang [9] further showed that FreeRTOS multitasking behavior can lead to complex timing and execution interactions. Nevertheless, this body of work generally assumes workloads with relatively well-defined and deterministic execution characteristics, rather than machine learning inference workloads whose runtime behavior must be considered within a broader embedded application context.

The missing intersection between these two areas defines the research gap addressed by this thesis. TinyML research often focuses on whether models can fit and run on embedded hardware, but it frequently overlooks the effects of multitasking, scheduling interference, and temporal variability. Real-time systems research, on the other hand, provides strong foundations for scheduling analysis but rarely considers embedded machine learning inference as a first-class workload. As a result, there is still limited understanding of whether an embedded ML model that is efficient in terms of size and average execution time will also remain temporally predictable once integrated into a real-time multitasking system. This gap justifies the present thesis, which investigates the timing behavior of embedded TinyML inference under realistic workload interference and scheduling conditions.

1.4 Research Objectives

This thesis aims to investigate the timing behavior of embedded TinyML inference in a real-time multitasking system. The central goal is to determine how scheduling decisions and concurrent workload interference influence inference latency, timing variability, and deadline compliance after deployment on resource-constrained embedded hardware.

The specific objectives of this study are as follows:

- To evaluate the effect of task priority on inference latency and deadline behavior.
- To characterize the jitter and execution-time variability of TinyML inference under multitasking conditions.
- To examine how competing sensing, communication, and input/output tasks influence inference timing.
- To identify scheduling conditions under which inference remains schedulable and conditions under which it becomes unschedulable.
- To determine whether model-level efficiency alone is an adequate indicator of real-time suitability in embedded deployment.

These objectives define the scope of the thesis and make clear that the study addresses not only whether embedded inference can run, but whether it can run predictably enough to satisfy real-time requirements.

1.5 Approach Overview

This thesis uses an experimental methodology to study the timing behavior of embedded TinyML inference in a multitasking real-time system. The evaluation is conducted on an ESP32-S3 platform running FreeRTOS to observe inference under conditions that more closely resemble practical embedded deployment than isolated benchmark execution.

The system is organized around a periodic inference task that performs on-device inference at regular intervals. To introduce controlled scheduling interference, the inference task runs concurrently with background workloads that emulate typical embedded functions, including sensing, communication, and input/output activity. This design enables examination of how FreeRTOS scheduling and competing workloads affect the temporal behavior of inference.

The study evaluates inference using four primary timing metrics: latency, jitter, observed worst-case execution time (WCET), and deadline misses. These measures are used to assess whether

inference remains predictable and temporally reliable under different multitasking conditions.

1.6 Contributions

The contributions of this thesis are summarized as follows:

- A practical experimental framework for evaluating embedded TinyML inference in a real-time multitasking environment on the ESP32-S3 platform using FreeRTOS.
- An experimental evaluation of periodic TinyML inference under controlled background workloads representing sensing, communication, and input/output activity.
- A characterization of how scheduling configuration and workload interference affect inference latency, jitter, observed worst-case execution time (WCET), and deadline-miss behavior.
- An analysis of the conditions under which embedded inference remains temporally acceptable and the conditions under which it becomes unschedulable or unreliable for real-time use.
- A system-level perspective showing that model-level efficiency alone is insufficient to judge the real-time suitability of embedded TinyML deployment.

1.7 Thesis Organization

The remainder of this thesis is organized as follows.

Chapter 2 presents the background and related work relevant to this study. It reviews embedded artificial intelligence, TinyML deployment on resource-constrained platforms, and real-time scheduling concepts needed to frame the thesis.

Chapter 3 describes the system design and methodology. It introduces the ESP32-S3 platform, the FreeRTOS-based execution model, the periodic inference task, the controlled background workloads, and the timing metrics used in the evaluation.

Chapter 4 presents the experimental setup. It describes the hardware and software environment, workload configurations, scheduling scenarios, and data collection procedure used to evaluate inference timing behavior.

Chapter 5 reports the experimental results and analysis. It compares inference timing across different scheduling configurations and examines the effects of workload interference on latency, jitter, observed worst-case execution time, and deadline misses.

Chapter 6 discusses the implications of the results. It interprets the findings from a real-time systems perspective, identifies conditions under which embedded inference remains acceptable or becomes unschedulable, and explains the broader significance of the gap between model-level efficiency and system-level predictability.

Chapter 7 concludes the thesis and outlines directions for future work. It summarizes the main findings and suggests possible extensions involving additional workloads, scheduling policies, platforms, and model types.

Chapter 2

BACKGROUND

This chapter presents the theoretical and technical background that supports the research problem addressed in this thesis. It introduces Embedded AI and TinyML to explain how machine learning models are adapted for execution on resource-constrained microcontrollers, then reviews the fundamentals of real-time embedded systems, including the distinction between hard and soft real-time requirements, the meaning of deadlines, and the importance of predictability and determinism. It also describes the FreeRTOS task model and scheduling behavior, highlighting how multiple tasks interact in a priority-based preemptive environment, and discusses common scheduling challenges in multitasking systems such as resource contention, starvation, priority imbalance, and non-deterministic execution. In addition, the chapter defines the key timing metrics used to evaluate real-time performance, including latency, jitter, worst-case execution time, deadline miss ratio, and lateness. Finally, it brings these ideas together by situating TinyML within practical embedded IoT systems, where inference operates alongside sensing, communication, and other background tasks, thereby establishing the foundation for analyzing the timing behavior of embedded machine learning under multitasking conditions.

2.1 Embedded AI and TinyML

Embedded Artificial Intelligence (Embedded AI) refers to the deployment of machine learning models directly on resource-constrained devices such as microcontrollers, sensors, and edge nodes. Unlike traditional cloud-based AI systems, embedded AI enables local data processing, reducing latency, bandwidth usage, and dependency on network connectivity. This paradigm has become increasingly important with the growth of Internet of Things (IoT) systems and real-time edge applications [1, 3].

Tiny Machine Learning (TinyML) is a specialized subset of embedded AI that focuses on

executing machine learning inference on ultra-low-power microcontrollers. These devices typically operate under strict constraints in memory, computational capability, and energy consumption. Compared to conventional machine learning deployment on servers or GPUs, TinyML targets systems with only tens to hundreds of kilobytes of RAM and limited processing power [8].

Traditional machine learning systems rely on high-performance computing platforms that support large models, floating-point operations, and complex runtime environments. In contrast, TinyML requires models to be compact, efficient, and optimized for constrained hardware. Techniques such as quantization, model compression, and architecture simplification are commonly applied to enable deployment within limited resources [1]. As a result, TinyML models are significantly smaller and more energy-efficient, enabling continuous operation on battery-powered or energy-harvesting devices.

2.1.1 Embedded Inference on Microcontrollers

Embedded inference refers to executing a trained machine learning model directly on a microcontroller without relying on external computation. In this approach, sensor data is processed locally, and predictions are generated in real time on-device. This eliminates the need for constant communication with cloud services, thereby reducing latency and improving system reliability.

Frameworks such as TensorFlow Lite for Microcontrollers provide lightweight runtime environments specifically designed for embedded systems. These frameworks use static memory allocation and minimal dependencies to ensure deterministic and efficient execution [7]. A typical embedded inference workflow includes:

- Sensor data acquisition (e.g., accelerometer, audio, or environmental signals)
- Preprocessing and feature extraction
- Model inference using a lightweight interpreter
- Decision-making or actuation based on model output

Unlike traditional systems, embedded inference must operate within strict real-time constraints, where execution latency and predictability are critical.

2.1.2 Resource Constraints in TinyML Systems

TinyML systems are fundamentally constrained by limited hardware resources:

Memory: Microcontrollers typically provide only a few kilobytes to a few hundred kilobytes of RAM, restricting model size and intermediate data storage.

Computation: These devices often lack hardware acceleration and have limited processing capabilities, requiring simplified operations and efficient implementations.

Power Consumption: Many TinyML applications are battery-powered, necessitating ultra-low-power operation for extended deployment lifetimes [13].

These constraints require careful co-design of both the machine learning model and the embedded system to ensure feasibility and efficiency.

2.1.3 TinyML Deployment Pipeline

The deployment of TinyML models follows a structured pipeline that differs from traditional machine learning workflows. It typically consists of three main stages:

Training: Model training is performed offline on powerful hardware such as GPUs or cloud platforms. During this stage, model parameters are learned using large datasets.

Quantization: The trained model is converted to a lower-precision representation (e.g., 8-bit integers) to reduce memory footprint and computational cost. This step is essential for deployment on microcontrollers.

Inference: The optimized model is deployed onto the target device, where it performs real-time predictions using incoming sensor data.

This process can be summarized as:

$$\text{Training} \rightarrow \text{Quantization} \rightarrow \text{Embedded Inference}$$

Overall, TinyML enables intelligent decision-making directly on embedded devices, forming the foundation for real-time applications such as activity recognition, anomaly detection, and autonomous sensing in constrained environments.

2.2 Real-Time Embedded Systems

Real-time embedded systems are computing systems in which the correctness of operation depends not only on the logical results of computations but also on the time at which those results are produced. In such systems, tasks must complete within specified timing constraints, known as deadlines, to ensure proper system behavior. These systems are widely used in domains such as industrial control, healthcare monitoring, automotive systems, and safety-critical applications [3, 2].

2.2.1 Hard and Soft Real-Time Systems

Real-time systems are typically classified into two categories: hard real-time and soft real-time systems.

Hard real-time systems require strict adherence to deadlines. Missing a deadline is considered a system failure and may lead to catastrophic consequences. Examples include avionics control systems, medical devices, and safety-critical industrial controllers. In these systems, timing guarantees must be provably satisfied under all conditions.

Soft real-time systems, on the other hand, tolerate occasional deadline misses, although such violations may degrade system performance or quality of service. Examples include multimedia streaming, human-computer interaction systems, and many IoT applications. While timely execution is important, occasional delays do not result in system failure.

In practice, many embedded systems fall between these two extremes, exhibiting characteristics of firm real-time systems, where results that miss deadlines are considered invalid but do not necessarily cause system-wide failure.

2.2.2 Deadlines and Timing Constraints

A deadline defines the maximum allowable time for a task to complete after it becomes ready for execution. In periodic real-time systems, tasks are typically characterized by three parameters: period, execution time, and deadline. The period defines how often a task is activated, while the execution time represents the required computation time. The deadline specifies when the task must finish relative to its activation.

Meeting deadlines requires careful scheduling of tasks, especially in multitasking environments where multiple tasks compete for limited processing resources. Scheduling policies must ensure that higher-priority or time-critical tasks receive sufficient CPU time to complete within their deadlines.

However, in practical systems, various factors such as resource contention, synchronization mechanisms, and interrupt handling can introduce delays and variability in execution time. For example, priority inversion can occur when a high-priority task is blocked by a lower-priority task holding a shared resource, leading to unbounded delays if not properly managed [11, 12].

2.2.3 Predictability and Determinism

A key requirement of real-time systems is predictability. Predictability refers to the ability to determine, in advance, whether all tasks will meet their deadlines under given system conditions. This requires not only knowledge of average execution time but also worst-case execution time (WCET), which represents the maximum possible time a task may take to complete.

Determinism is closely related and refers to the consistency of system behavior across repeated executions. In a deterministic system, the same inputs and conditions produce the same timing behavior, allowing developers to reason about system performance with confidence.

In embedded operating systems such as FreeRTOS, scheduling behavior is designed to be predictable, typically using fixed-priority preemptive scheduling. However, real-world implementations may still exhibit timing variability due to system overheads, task interactions, and resource contention. Studies have shown that scheduling overhead and context-switching costs can impact

the timing behavior of periodic tasks, particularly under high system load [10, 9].

2.2.4 Implications for Embedded AI Systems

The integration of machine learning inference into real-time embedded systems introduces additional challenges. Unlike traditional control tasks, inference workloads may exhibit variable execution times depending on model complexity and input data. This variability can affect system predictability and increase the risk of deadline misses.

Therefore, understanding real-time scheduling principles and timing behavior is essential when deploying TinyML models in embedded environments. Ensuring that inference tasks meet timing constraints requires careful consideration of task priorities, system load, and resource allocation.

This foundation is critical for analyzing how scheduling decisions impact inference latency, jitter, and deadline behavior in multitasking embedded systems, which is the focus of this work.

2.3 FreeRTOS Task Model and Scheduling

FreeRTOS is a widely used real-time operating system (RTOS) designed for embedded systems. It provides a lightweight and deterministic execution environment for managing multiple concurrent tasks on resource-constrained hardware. The FreeRTOS task model is based on priority-driven scheduling, where multiple tasks share CPU time according to their assigned priorities and system state [9].

2.3.1 Task Abstraction

In FreeRTOS, a task represents an independent unit of execution, similar to a thread in general-purpose operating systems. Each task has its own execution context, including a stack, program counter, and local variables. Tasks are created to perform specific functions, such as sensor acquisition, communication, input/output operations, or computation.

Tasks in a system typically execute concurrently, but on a single-core microcontroller, only one task can run at any given time. The RTOS scheduler is responsible for determining which task is executed based on system conditions and task priorities.

2.3.2 Task States

At any point in time, a FreeRTOS task exists in one of several states:

- **Running:** The task is currently executing on the CPU.
- **Ready:** The task is prepared to execute and is waiting to be scheduled.
- **Blocked:** The task is waiting for an event, such as a delay timeout, resource availability, or external input.

The scheduler transitions tasks between these states based on timing events, synchronization mechanisms, and system calls. For example, a task that calls a delay function enters the blocked state until the specified time elapses, after which it returns to the ready state.

2.3.3 Priority-Based Preemptive Scheduling

FreeRTOS uses a fixed-priority preemptive scheduling policy. Each task is assigned a priority level, and the scheduler always selects the highest-priority task in the ready state for execution.

When a higher-priority task becomes ready (e.g., due to an interrupt or event), it immediately preempts the currently running lower-priority task. This ensures that time-critical tasks can respond promptly to events, which is essential for real-time system behavior.

If multiple tasks share the same priority, FreeRTOS can optionally use time slicing to distribute CPU time among them. In this case, tasks of equal priority are executed in a round-robin fashion, each receiving a time slice before yielding to the next.

2.3.4 CPU Time Allocation and Task Interaction

CPU time in FreeRTOS is allocated dynamically based on task priority and state. Higher-priority tasks dominate CPU access, while lower-priority tasks execute only when higher-priority tasks are either blocked or not ready.

This behavior leads to important interactions between tasks:

- **Preemption:** Higher-priority tasks can interrupt lower-priority tasks at any time.
- **Blocking:** Tasks voluntarily relinquish the CPU when waiting for events or delays.
- **Contention:** Multiple tasks may compete for CPU time or shared resources.

In systems with heavy workload or poorly balanced priorities, lower-priority tasks may experience starvation, where they rarely or never get CPU time. Additionally, synchronization between tasks can introduce delays due to resource contention and scheduling overhead [10].

2.3.5 Implications for Multitasking Systems

The FreeRTOS task model enables efficient multitasking on embedded devices, but it also introduces complexity in timing behavior. Since task execution depends on priority, system load, and interaction with other tasks, the actual execution time of a task may vary significantly.

This variability is particularly important in systems that combine computation-intensive tasks with real-time requirements. For example, when multiple tasks such as sensing, communication, and data processing execute concurrently, scheduling decisions directly affect latency, jitter, and deadline satisfaction.

Understanding how tasks transition between states and compete for CPU time is therefore essential for analyzing system performance. This is especially relevant when integrating machine learning inference into real-time systems, where execution timing must remain predictable despite additional computational load.

2.4 Scheduling Challenges in Multitasking Systems

In multitasking real-time systems, multiple tasks execute concurrently while sharing limited computational resources. Although real-time operating systems such as FreeRTOS provide structured scheduling mechanisms, several challenges arise that can significantly affect timing behavior. These challenges introduce variability in execution time and complicate the ability to guarantee deadline satisfaction.

2.4.1 Resource Contention

Resource contention occurs when multiple tasks compete for shared system resources, such as the CPU, memory, communication interfaces, or peripheral devices. Since only one task can utilize the CPU at a given time, tasks must wait for access when higher-priority or currently running tasks occupy the processor.

Contention can also arise when tasks share synchronization primitives such as mutexes or semaphores. In such cases, a task may be blocked while waiting for a resource to be released by another task. This waiting time contributes to increased response time and may delay task completion.

In real-time systems, resource contention directly impacts latency and can lead to increased variability in execution time. Under heavy system load, contention becomes more pronounced, making it difficult to predict task completion times accurately.

2.4.2 Starvation

Starvation occurs when a task is unable to obtain sufficient CPU time to execute, typically due to the continuous execution of higher-priority tasks. In priority-based scheduling systems like FreeRTOS, lower-priority tasks are only scheduled when higher-priority tasks are not in the ready state.

If higher-priority tasks execute frequently or do not yield the CPU, lower-priority tasks may

experience prolonged delays or may never execute at all. This can lead to missed deadlines and incomplete system functionality.

Starvation is particularly problematic in systems with imbalanced priority assignments, where certain tasks dominate CPU usage. It represents an extreme case of scheduling unfairness and highlights the importance of careful priority configuration.

2.4.3 Priority Imbalance

Priority imbalance refers to suboptimal assignment of task priorities, where priorities do not accurately reflect task urgency or timing requirements. In such cases, tasks that are less time-critical may preempt more critical tasks, leading to inefficient CPU utilization and increased deadline misses.

Additionally, improper priority assignment can exacerbate other scheduling issues, such as starvation and priority inversion. Priority inversion occurs when a high-priority task is indirectly blocked by a lower-priority task holding a shared resource, potentially leading to unbounded delays if mitigation mechanisms are not applied [11, 12].

Effective priority assignment is therefore essential for maintaining system responsiveness and ensuring that critical tasks meet their timing constraints.

2.4.4 Non-Deterministic Execution

Non-deterministic execution refers to variability in task execution timing across different runs of the system. Even when the task code remains unchanged, factors such as scheduling decisions, interrupt timing, resource contention, and system overhead can cause execution time to fluctuate.

This variability manifests as jitter, which is the deviation in task execution timing from its expected schedule. In real-time systems, high jitter reduces predictability and increases the likelihood of deadline violations.

Sources of non-determinism include context-switch overhead, interrupt handling, and in-

teractions between concurrently executing tasks. Studies have shown that such overheads can significantly influence the timing behavior of periodic tasks in embedded systems [10, 9].

2.4.5 Impact on Timing Behavior

The combined effect of resource contention, starvation, priority imbalance, and non-deterministic execution leads to complex timing behavior in multitasking systems. These factors contribute to:

- Increased latency due to delayed task execution
- Variability in execution time (jitter)
- Occurrence of deadline misses under high system load

As system complexity increases, these effects become more pronounced, making it difficult to guarantee real-time performance without careful system design and analysis.

2.4.6 Relevance to Embedded AI Workloads

When machine learning inference is integrated into multitasking embedded systems, these scheduling challenges become even more significant. Inference tasks introduce additional computational demand, increasing competition for CPU resources and amplifying timing variability.

Understanding these challenges is essential for analyzing how scheduling configurations influence inference latency, jitter, and deadline behavior. This forms the basis for the experimental evaluation presented in later chapters, where different priority configurations are used to study the interaction between inference tasks and background workloads.

2.5 Timing Metrics in Real-Time Systems

Evaluating the performance of real-time systems requires quantitative metrics that capture both correctness and timing behavior. In addition to producing correct outputs, tasks must execute

within specified time constraints. Several key timing metrics are commonly used to characterize system performance, including latency, jitter, worst-case execution time (WCET), deadline miss ratio, and lateness.

2.5.1 Latency

Latency refers to the time elapsed between the activation of a task and the completion of its execution. In real-time systems, this typically represents the delay from when a task becomes ready (e.g., triggered by a periodic timer or event) to when its output is produced.

Latency is a fundamental measure of responsiveness. Lower latency indicates that the system can react more quickly to inputs or events. However, in multitasking environments, latency can increase due to scheduling delays, resource contention, and preemption by higher-priority tasks.

2.5.2 Jitter

Jitter describes the variability in task execution timing across multiple activations. Even if a task has a fixed period, the actual time at which it starts or completes may vary from one instance to another.

High jitter indicates inconsistent timing behavior, which can be problematic in real-time systems that require regular and predictable execution. For example, in control systems or signal processing applications, irregular timing may degrade system stability or output quality.

Jitter is therefore an important indicator of system predictability and temporal stability.

2.5.3 Worst-Case Execution Time (WCET)

Worst-case execution time (WCET) is the maximum amount of time a task may take to complete under any possible execution scenario. Unlike average execution time, WCET represents the upper bound that must be considered when verifying whether deadlines can be met.

Accurate estimation of WCET is critical in hard real-time systems, where missing a deadline is unacceptable. WCET must account for all sources of delay, including computation time,

scheduling overhead, and interference from other tasks.

In practice, determining exact WCET can be challenging due to system complexity and non-deterministic factors. As a result, systems often rely on observed or estimated upper bounds to approximate WCET.

2.5.4 Deadline Miss Ratio

The deadline miss ratio represents the proportion of task instances that fail to complete before their assigned deadlines. It is typically expressed as a percentage of total task activations.

This metric provides a direct measure of system schedulability and reliability. A low or zero deadline miss ratio indicates that the system is able to meet its timing requirements consistently, while a high miss ratio suggests that the system is overloaded or poorly configured.

In soft real-time systems, occasional deadline misses may be acceptable, but their frequency must be controlled to maintain acceptable performance.

2.5.5 Lateness

Lateness quantifies how much a task exceeds its deadline when a deadline miss occurs. It is defined as the difference between the actual completion time and the assigned deadline.

Lateness provides additional insight beyond the binary notion of deadline success or failure. Two systems with the same deadline miss ratio may exhibit very different behaviors if one system produces small timing violations while another produces large delays.

Therefore, lateness is useful for understanding the severity of deadline violations and their potential impact on system performance.

2.5.6 Summary and Relevance

These timing metrics collectively provide a comprehensive view of real-time system behavior. Latency captures responsiveness, jitter reflects variability, WCET defines execution bounds, deadline miss ratio measures reliability, and lateness characterizes the severity of timing violations.

Together, they form the basis for analyzing and evaluating the performance of multitasking embedded systems. In the context of this work, these metrics are used to assess how scheduling configurations and workload interactions influence the timing behavior of embedded machine learning inference tasks.

2.6 TinyML in Embedded IoT Systems

In practical deployments, TinyML does not operate in isolation but as part of a larger embedded system that integrates sensing, communication, and control functionalities. These systems are typically built on microcontrollers running real-time operating systems, where multiple tasks execute concurrently and compete for limited computational resources.

A typical embedded IoT system consists of several interacting components:

- **Sensing tasks:** responsible for acquiring data from sensors such as accelerometers, microphones, or environmental sensors
- **Inference tasks:** responsible for executing machine learning models on collected data
- **Communication tasks:** responsible for transmitting data or results using protocols such as MQTT or other network interfaces
- **I/O and system tasks:** responsible for logging, storage, and device management

These tasks operate concurrently under the control of an RTOS scheduler, such as FreeRTOS, and interact through shared resources and timing constraints. As a result, the execution of any single task, including machine learning inference, is influenced by the behavior of other tasks in the system.

2.6.1 Inference as a System-Level Task

In many TinyML applications, inference is implemented as a periodic task that processes sensor data at regular intervals. For example, an activity recognition system may perform inference

every few milliseconds using buffered sensor readings. This periodic behavior introduces explicit timing requirements, where each inference cycle must complete within a defined deadline.

Unlike standalone machine learning benchmarks, embedded inference must operate within a multitasking environment. The inference task must share CPU time with sensing, communication, and other background activities. Consequently, its execution timing is subject to scheduling delays, preemption, and resource contention.

This interaction means that the performance of inference cannot be evaluated solely based on model efficiency. Instead, it must be analyzed within the context of the entire system, where scheduling decisions and workload interactions directly affect latency, jitter, and deadline behavior.

2.6.2 Interaction with Communication and I/O Workloads

Communication and I/O tasks introduce additional complexity into embedded IoT systems. Data transmission protocols, such as MQTT, may involve buffering, packet handling, and network delays, all of which consume CPU time and system resources [4, 5]. Similarly, logging and storage operations can introduce blocking behavior, further affecting task scheduling.

These workloads often exhibit bursty or irregular execution patterns, which can interfere with the periodic execution of inference tasks. For example, a communication task handling incoming or outgoing messages may temporarily preempt other tasks, increasing inference latency or causing variability in execution timing.

As system load increases, the combined effect of sensing, communication, and I/O activities can significantly degrade timing predictability, particularly in systems with limited computational capacity.

2.6.3 Implications for Real-Time Performance

The integration of TinyML into embedded IoT systems introduces a complex interplay between computation and scheduling. Inference tasks must coexist with other system activities while still meeting real-time constraints. This creates several challenges:

- Increased latency due to competition for CPU resources
- Higher jitter caused by variable scheduling delays
- Potential deadline misses under heavy workload conditions

These challenges highlight the importance of analyzing inference performance within realistic multitasking environments rather than isolated conditions.

2.6.4 Bridge to This Work

Given that TinyML inference operates as one task among many in a real-time embedded system, understanding its timing behavior requires a system-level perspective. The interaction between task scheduling, workload characteristics, and resource constraints plays a critical role in determining whether inference can meet real-time requirements.

This work investigates how different scheduling configurations affect the timing behavior of embedded inference tasks. By evaluating metrics such as latency, jitter, and deadline miss ratio under controlled multitasking conditions, this study aims to characterize the conditions under which TinyML inference remains schedulable and identify scenarios that lead to degraded real-time performance.

Chapter 3

Related Works

The integration of machine learning into resource-constrained embedded systems has led to growing research interest across multiple domains, including TinyML model optimization, embedded AI applications, and real-time system design. While significant progress has been made in each of these areas, the challenges associated with deploying machine learning inference within multitasking real-time environments remain insufficiently addressed. In particular, existing studies tend to examine model efficiency, system scheduling, and application-level behavior in isolation, without fully considering their combined impact on timing performance. This chapter reviews the relevant literature in these domains, with the aim of identifying key limitations and establishing the need for a system-level investigation of TinyML inference under realistic embedded workloads.

3.1 Real-Time Scheduling in Embedded Systems

Real-time embedded systems are designed to guarantee the timely execution of tasks under strict timing constraints. Unlike general-purpose systems, correctness in real-time systems depends not only on functional output but also on when the output is produced. Tasks must be completed within predefined deadlines, and failure to do so can lead to degraded system performance or critical failures.

Most real-time operating systems (RTOS), including FreeRTOS, employ *priority-based preemptive scheduling*. In this model, each task is assigned a priority level, and the scheduler always selects the highest-priority ready task for execution. Lower-priority tasks can be preempted whenever a higher-priority task becomes ready, enabling responsive handling of time-critical operations [9].

However, this scheduling approach introduces several well-known challenges. One fundamental issue is *priority inversion*, where a high-priority task is indirectly blocked by a lower-priority

task holding a shared resource. This situation can be exacerbated by medium-priority tasks that preempt the lower-priority task, further delaying the execution of the higher-priority task. Early work by Sha *et al.* introduced the Priority Inheritance Protocol (PIP) as a solution, allowing the lower-priority task to temporarily inherit the higher priority to resolve the inversion [11]. Subsequent studies have shown that unbounded priority inversion can still occur under certain conditions, highlighting the complexity of resource synchronization in real-time systems [12].

In addition to synchronization issues, scheduling overhead is another important factor affecting system performance. Context switching, task management, and interrupt handling introduce additional execution time that can impact the schedulability of periodic tasks. Experimental analysis of FreeRTOS has shown that these overheads, while relatively small, become significant in systems with tight timing constraints or high task frequencies [10].

Real-world embedded systems rarely consist of a single task. Instead, they involve multiple concurrent activities such as sensing, communication, control, and data processing. In IoT-based applications, communication protocols such as MQTT introduce additional workload variability due to network delays, message handling, and asynchronous events [4, 5, 6]. Similarly, applications in domains such as public safety and emergency response require coordination of multiple real-time tasks operating under dynamic conditions [3, 2].

These factors lead to complex interactions between tasks, where execution timing becomes non-deterministic due to contention for CPU time and shared resources. As a result, even if individual tasks are designed to meet their timing requirements in isolation, their combined behavior may lead to deadline misses, increased latency, and timing variability.

While prior research has extensively analyzed scheduling behavior and synchronization mechanisms, most studies focus on traditional workloads such as control tasks, communication processes, or synthetic benchmarks. The interaction between modern workloads—such as TinyML inference—and real-time scheduling remains less explored. In particular, the impact of priority assignment, workload interference, and system-level contention on inference timing has not been thoroughly characterized.

This gap motivates the need to study TinyML inference not as an isolated computation, but as part of a multitasking real-time system, where scheduling decisions directly influence its timing behavior.

3.2 Embedded AI Applications in IoT

The integration of artificial intelligence into embedded and Internet of Things (IoT) systems has enabled a wide range of intelligent applications at the network edge. By performing data processing and inference locally on resource-constrained devices, embedded AI systems reduce reliance on cloud infrastructure, enabling lower latency, improved privacy, and more efficient use of network bandwidth [1].

A common class of embedded AI applications is *monitoring and anomaly detection*. In industrial and infrastructure settings, machine learning models are deployed on edge devices to continuously analyze sensor data and detect abnormal conditions. These systems can identify early signs of equipment failure, environmental hazards, or operational inefficiencies, allowing for timely intervention without requiring constant data transmission to centralized servers. Similar approaches are used in smart home and health monitoring systems, where embedded models process data from sensors to detect unusual patterns and trigger alerts [2].

Another important use case is *smart sensing*, where machine learning enhances the capability of traditional sensors by enabling higher-level interpretation of raw data. For example, embedded vision and audio models can perform tasks such as object detection, gesture recognition, and speech classification directly on-device. This allows systems to operate autonomously and respond in real time to environmental inputs. Frameworks such as TensorFlow Lite for Microcontrollers have made it feasible to deploy such models on low-power microcontrollers with limited memory and computational resources [7, 8].

Embedded AI is also widely used in *communication-aware IoT systems*, where local inference is combined with network protocols to support distributed decision-making. Applications such as public safety monitoring and emergency response systems rely on edge intelligence to

process sensor data, prioritize events, and communicate relevant information efficiently across the network [3]. In these systems, the interaction between computation and communication introduces additional complexity, as both processing delays and network latency contribute to overall system responsiveness.

Despite the growing adoption of embedded AI in IoT applications, most existing studies evaluate performance at a high level, focusing on metrics such as accuracy, throughput, or general responsiveness. While latency is often mentioned as an important factor, it is typically reported as an average or aggregate value, without a detailed characterization of timing behavior under realistic operating conditions. In particular, metrics such as jitter, worst-case execution time (WCET), and deadline miss ratio are rarely analyzed in depth.

Furthermore, many embedded AI evaluations assume simplified execution environments in which inference runs with minimal interference from other tasks. In practical IoT systems, however, embedded devices must concurrently handle sensing, communication, and I/O operations, all competing for limited processing resources. These interactions can introduce variability in execution timing that is not captured by conventional performance evaluations.

This limitation highlights a gap in the literature: while embedded AI applications demonstrate the feasibility and utility of on-device intelligence, there is limited understanding of how these systems behave under real-time multitasking conditions. Addressing this gap requires a system-level perspective that considers not only model performance but also the impact of scheduling and workload interactions on timing behavior.

3.3 Real-Time Scheduling in Embedded Systems

Real-time embedded systems rely on scheduling policies to determine the order in which tasks execute and whether timing constraints can be satisfied. In these systems, correctness depends not only on producing the correct functional result but also on producing that result within an acceptable time bound. As a result, the behavior of the scheduler plays a central role in overall system performance and predictability.

Most real-time operating systems use *priority-based scheduling*, in which each task is assigned a priority, and the highest-priority ready task is selected for execution. In preemptive systems, a newly ready higher-priority task can interrupt a currently running lower-priority task. This improves responsiveness for time-critical activities, but it also means that execution order depends on task release patterns, blocking conditions, and resource availability. In practice, task priorities strongly influence response time, interference, and deadline satisfaction.

A substantial body of research has examined scheduling and synchronization issues in real-time systems. Early foundational work on priority inversion showed that when tasks share resources, a higher-priority task can be delayed by a lower-priority one, potentially causing unpredictable blocking. Protocols such as priority inheritance and priority ceiling were proposed to reduce this effect and improve schedulability [11]. Related work further analyzed sources of unbounded priority inversion and emphasized that synchronization and communication activities can significantly disrupt timing behavior if not properly controlled [12].

More recent studies have examined the behavior of embedded RTOS schedulers directly. Formal analysis of the FreeRTOS scheduler has shown that task interleavings, interrupt behavior, and scheduling policy choices can lead to complex execution patterns, including starvation and assertion failures that are difficult to detect through conventional testing alone [9]. Other work has focused on quantifying RTOS overheads, showing that kernel activities such as tick handling and context switching contribute additional execution time and may affect periodic task response times, especially in systems with tight timing requirements [10].

Taken together, these studies show that scheduling affects embedded execution in two fundamental ways. First, it determines *execution order* by deciding which ready task runs at a given moment. Second, it affects *timing behavior* by introducing interference, blocking, and overhead that alter when a task actually begins and completes execution. Even when individual tasks are functionally correct, their timing can vary substantially depending on priority assignment and concurrent workload conditions.

However, most prior scheduling studies analyze systems composed of conventional embed-

ded workloads, such as periodic control tasks, communication routines, interrupt-driven services, and resource-sharing processes. These works generally assume workloads whose execution demand is relatively predictable or can be bounded using traditional real-time analysis techniques. By contrast, machine learning inference introduces a different type of workload. Although inference is deterministic for a fixed model and input, its execution is typically treated at the application level rather than as a classical real-time task, and its interaction with competing sensing, communication, and I/O activities has received far less attention.

Thus, existing scheduling literature provides valuable insight into RTOS behavior, task prioritization, and timing interference, but it does not fully address the challenges posed by embedded ML inference in multitasking systems. In particular, these works largely assume predictable workloads, unlike ML inference, which motivates further study of how inference behaves when scheduled alongside other real-time embedded tasks.

3.4 Timing Analysis and WCET Estimation

Timing analysis is a fundamental aspect of real-time system design, ensuring that all tasks can meet their deadlines under worst-case conditions. A key concept in this domain is the *worst-case execution time* (WCET), which represents the maximum time a task may take to complete execution on a given platform. Accurate WCET estimation is essential for schedulability analysis, where system designers verify that all tasks can execute within their deadlines under the chosen scheduling policy.

Traditional approaches to WCET estimation fall into two main categories: *static analysis* and *measurement-based analysis*. Static analysis techniques examine the program code and hardware characteristics to derive an upper bound on execution time without executing the program. These methods consider factors such as control flow, instruction timing, and pipeline behavior. In contrast, measurement-based approaches rely on empirical observation, where execution times are recorded under various input conditions, and the maximum observed value is used as an estimate of WCET [14].

Both approaches typically assume a degree of *predictability* in task execution. This includes assumptions such as deterministic control flow, bounded loop iterations, and stable hardware behavior. In many traditional embedded applications—such as control systems or signal processing pipelines—these assumptions are reasonable, allowing WCET to be estimated with sufficient confidence for real-time guarantees.

However, modern embedded systems increasingly incorporate more complex workloads that challenge these assumptions. In particular, machine learning inference introduces characteristics that differ from conventional real-time tasks. While inference execution for a fixed model is deterministic at the instruction level, its overall timing behavior can exhibit variability due to factors such as memory access patterns, cache effects, and interactions with other concurrent tasks.

Furthermore, ML inference is often integrated into multitasking systems where it competes with sensing, communication, and I/O operations for shared resources. This introduces additional variability in execution timing that is not captured by traditional WCET models. Measurement-based approaches may underestimate worst-case conditions if they do not account for all possible interference scenarios, while static analysis becomes difficult due to the complexity of modern inference frameworks and hardware platforms.

As a result, there is a mismatch between traditional timing analysis techniques and the behavior of embedded ML workloads:

- ML inference introduces execution-time variability due to system-level interactions and resource contention.
- Traditional WCET estimation methods rely on predictable execution patterns and may not directly apply to such workloads.

This gap highlights the need for empirical, system-level evaluation of timing behavior in embedded ML applications, where latency, jitter, and deadline adherence must be analyzed under realistic multitasking conditions rather than assumed from isolated execution.

3.5 Summary and Research Gap

This chapter has reviewed three major areas relevant to this work: TinyML model optimization, embedded AI applications in IoT systems, and real-time scheduling and timing analysis in embedded environments. While each of these domains has been extensively studied, they have largely evolved independently, leading to important gaps when considering their integration in practical systems.

First, research in TinyML has primarily focused on improving model efficiency. Techniques such as compression, quantization, and hardware-aware optimization aim to reduce memory usage, computational cost, and energy consumption. These efforts have enabled the deployment of machine learning models on resource-constrained microcontrollers. However, they typically evaluate performance in isolation, without considering how inference behaves when executed alongside other tasks in a real-time system.

Second, real-time systems research has concentrated on scheduling policies, task prioritization, and synchronization mechanisms. These studies provide valuable insights into how task interactions, priority inversion, and system overhead affect execution order and timing guarantees. However, they are generally based on traditional embedded workloads with predictable execution characteristics and do not explicitly consider the behavior of machine learning inference as part of the workload.

Third, existing work on embedded AI applications demonstrates the feasibility of deploying intelligent functionality at the edge, including monitoring, anomaly detection, and smart sensing. While latency is often recognized as an important performance metric, most studies report it at a high level and do not provide a detailed analysis of timing behavior. In particular, there is limited investigation of variability-related metrics such as jitter, worst-case execution time, and deadline miss ratio under realistic multitasking conditions.

As a result, there is a lack of comprehensive experimental studies that examine how TinyML inference behaves within a real-time embedded system where multiple tasks compete for shared

resources. The interaction between scheduling policies, workload interference, and inference execution remains insufficiently understood.

This thesis addresses this gap by experimentally evaluating the impact of scheduler-level interference on TinyML inference timing.

Chapter 4

System Design

This chapter describes the design of the embedded TinyML inference system developed for this study. It outlines the overall system organization and explains how its hardware and software elements are integrated to enable real-time inference on a resource-constrained platform. Particular attention is given to the implementation on the ESP32-S3 under FreeRTOS, including the inference workflow, task structure, communication and logging support, and the timing observation mechanisms used during execution. The system design is informed by prior studies on TensorFlow Lite Micro, embedded AI deployment, and real-time scheduling in FreeRTOS-based environments [7, 8, 1, 9, 10]. The following subsections describe the system architecture, software structure, task organization, and timing instrumentation in detail.

4.1 Design Goals and Requirements

The design of the proposed system is guided by the following goals and requirements:

- **Periodic on-device inference:** The system must support the periodic execution of TinyML inference so that model evaluation reflects the behavior of real-time embedded sensing applications.
- **Coexistence with concurrent tasks:** Inference must operate alongside other embedded activities, such as sensing, communication, and I/O processing, in order to represent a realistic multitasking environment.
- **Controllable scheduler interference:** The system must allow the interference imposed on the inference task to be adjusted in a controlled manner through task priorities, activation patterns, and workload intensity.

- **Timing observability:** The execution flow must support accurate observation of runtime timing behavior so that inference performance can be measured under different scheduling conditions.
- **Lightweight embedded deployment:** The implementation must remain compact enough to execute reliably on the ESP32-S3 within the compute and memory constraints of an embedded platform.
- **Modular task decomposition:** The system is organized into separate functional tasks so that inference, workload generation, and logging behavior can be isolated, configured, and analyzed systematically.
- **Single-core execution for controlled scheduling analysis:** Although the ESP32-S3 provides multiple cores, the experimental design constrains execution in a way that allows scheduler interactions to be examined in a more controlled and interpretable manner.
- **Fine-grained instrumentation for reproducible timing study:** The design incorporates detailed timing capture and structured logging so that execution behavior can be recorded consistently and analyzed across repeated experiments.

4.2 Target Platform and Software Environment

4.2.1 Hardware Platform

The target platform for the proposed system is the ESP32-S3, which serves as the embedded execution environment for both the TinyML inference task and the concurrent background workloads. From a design perspective, the ESP32-S3 is well suited to this study because it provides sufficient embedded-scale compute capability to execute a quantized neural network model while still operating under the practical resource constraints typical of microcontroller-based systems. This balance makes it an appropriate platform for examining how on-device inference behaves when integrated with real-time task scheduling and competing system activity.

A key reason for selecting the ESP32-S3 is that it represents a realistic TinyML deployment target rather than an oversized evaluation platform. Its limited RAM and storage require careful management of model size, tensor memory, task structure, and logging overhead. These limitations directly shape the system design, influencing how inference is implemented, how supporting workloads are constructed, and how runtime measurements are recorded. In this sense, the constraints of the ESP32-S3 are not merely background characteristics of the hardware but an essential part of the research motivation. They provide the conditions under which the interaction between TinyML inference and real-time scheduling can be studied meaningfully in an embedded setting.

4.2.2 Software Stack

The software environment is organized as a layered stack that supports platform integration, real-time scheduling, embedded inference, kernel-level acceleration, and persistent timing storage. At the foundation of this stack is the ESP-IDF framework, which provides the development environment, build system, hardware abstraction, peripheral support, and runtime services required for the ESP32-S3. Within this environment, FreeRTOS provides the core real-time execution model, managing task creation, priority-based preemptive scheduling, timing control, and task synchronization.

On top of the operating system layer, TensorFlow Lite Micro (TFLM) is used to execute the deployed TinyML model on the microcontroller. TFLM provides the embedded inference engine, including model loading, tensor allocation, interpreter management, and invocation of the quantized neural network. To improve inference efficiency on Espressif hardware, the system also uses `esp-nn`, which supplies optimized neural network kernels for supported operations and reduces computational overhead during inference. Finally, SPIFFS and the associated logging support provide persistent on-device storage for timing records and runtime summaries, allowing experimental outputs to be retained for later analysis.

These layers operate together as an integrated execution environment. ESP-IDF provides the system-level framework and hardware access. FreeRTOS schedules the inference task and

the background workload tasks according to the configured priorities and activation behavior. TFLM performs the actual forward execution of the deployed model, while `esp-nn` accelerates supported low-level operations to improve inference efficiency on the target platform. The timing and logging components then record the resulting runtime behavior to SPIFFS. Together, this stack establishes the complete software environment used to evaluate TinyML inference under controlled multitasking and scheduling conditions. The main layers of this software stack and their functions are summarized in Table 4.1.

Table 4.1: Software Stack of the Proposed System

Layer Component	**Role in the System**	**Main Function**	**Relevance to This Study**
ESP-IDF	Platform framework and hardware integration	Provides build tools, hardware abstraction, peripheral support, and runtime services for the ESP32-S3	Forms the overall embedded software environment in which the system is implemented and deployed
FreeRTOS	Real-time operating system layer	Manages task creation, preemptive priority-based scheduling, timing control, and multitasking behavior	Enables controlled study of scheduler interference and concurrent task execution
TensorFlow Lite Micro (TFLM)	Embedded inference engine	Loads the model, allocates tensors, manages the interpreter, and executes on-device inference	Provides the TinyML execution framework used for runtime inference on the microcontroller
`esp-nn`	Kernel-level inference acceleration	Supplies optimized neural network kernels for supported operations on Espressif hardware	Reduces inference overhead and improves the practicality of embedded TinyML deployment
SPIFFS / Logging Module	Persistent storage and measurement recording	Stores timing samples, raw logs, and summary outputs generated during experiments	Supports reproducible timing analysis by preserving runtime measurements for later evaluation

4.3 Overall System Architecture

The embedded TinyML inference framework is organized as a modular task-based system implemented on the ESP32-S3 under FreeRTOS. Rather than treating inference as an isolated benchmark, the design places the inference engine within a multitasking embedded environment in which scheduling, workload interference, timing measurement, and data logging operate together. This organization allows the system to reflect a more realistic deployment setting while still preserving the control needed for systematic timing evaluation.

The architecture consists of five main functional blocks. The first is the **inference engine**, which contains the deployed TensorFlow Lite Micro model, the interpreter, and the statically allocated tensor arena required for model execution. This block is responsible for preparing the input tensor, invoking inference, and producing the final classification output. The second block is the **scheduler and task management layer**, provided by FreeRTOS, which governs the execution of the inference task and all concurrent workload tasks according to their priority and readiness state. The third block is the set of **background workloads**, which includes sensor, communication, and I/O tasks designed to emulate common embedded activities and generate controlled scheduling interference. The fourth block is the **timing instrumentation module**, which captures execution timestamps at selected points in the inference cycle so that runtime behavior can be observed with fine granularity. The fifth block is the **logging and output path**, which records raw timing samples and summary information for later analysis through the on-device file system.

This modular decomposition is central to the design of the system. Each functional element is separated clearly enough to support controlled experimentation, yet all components remain integrated within a single embedded runtime environment. As a result, the framework supports reproducible evaluation of how TinyML inference behaves under multitasking and scheduler-level interference.

Figure 4.1 illustrates the overall system architecture. It shows the ESP32-S3 platform, the FreeRTOS scheduler, the dedicated inference task, the sensor, communication, and I/O workload

tasks, the timing recorder and logger, and the model execution resources, including the interpreter and tensor arena.

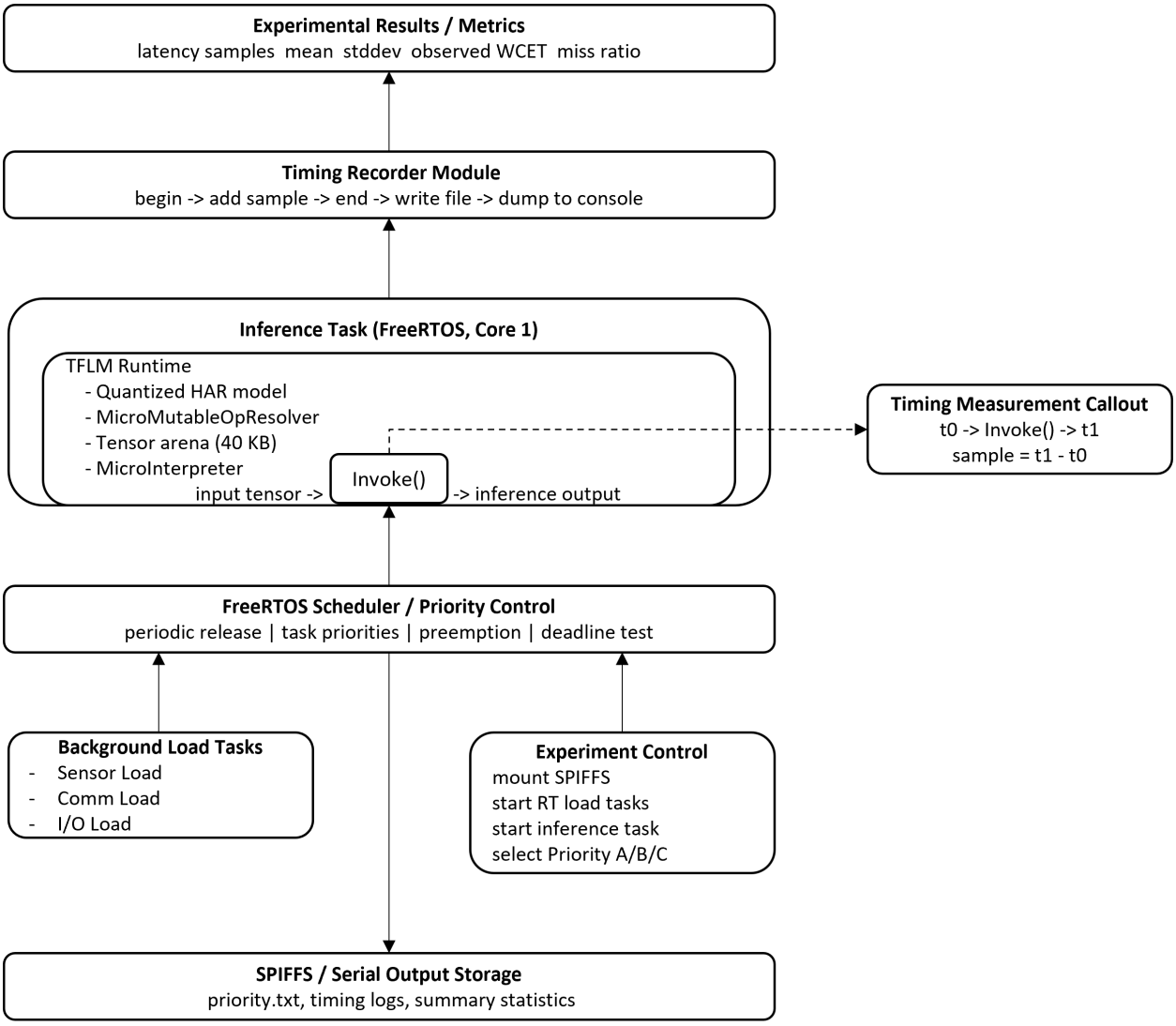


Figure 4.1: System Architecture of the Embedded TinyML Inference Framework. The ESP32-S3 executes the inference task under FreeRTOS scheduling alongside sensor, communication, and I/O workload tasks, with timing instrumentation and logging support for runtime performance evaluation.

4.4 TinyML Model and Inference Pipeline

The deployed model is based on the UCI Human Activity Recognition (HAR) dataset [15]. It is designed for embedded execution and performs multi-class classification of human activities

using inertial sensor data. The model input is a 128×6 time-series window, where 128 represents the number of temporal samples, and 6 corresponds to the six sensor channels. The output is a six-class prediction representing the target activities: walking, walking upstairs, walking downstairs, sitting, standing, and lying. To reduce memory usage and improve execution efficiency on the ESP32-S3, the model is converted to an INT8 quantized format, so that both weights and activations are represented using 8-bit integers.

From an architectural perspective, the model follows a lightweight one-dimensional convolutional neural network (1D CNN) structure suitable for microcontroller deployment. It consists of two Conv1D layers with 16 and 32 filters, respectively, each using a kernel size of 5 and ReLU activation, followed by MaxPooling1D layers for temporal downsampling. The extracted feature maps are then flattened and passed to a fully connected dense layer with 64 neurons, followed by a softmax output layer with 6 neurons. The model contains 69,078 trainable parameters in total. Inference is executed using TensorFlow Lite Micro (TFLM), with memory managed through a statically allocated tensor arena [8, 7]. In the current implementation, a 40 KB tensor arena is reserved to accommodate model tensors and intermediate activations during inference.

Within the system, the model is executed by a dedicated inference task. Each activation of the inference task performs one complete inference cycle, which consists of the following stages:

1. **Model initialization:** During system startup, the TFLM interpreter is created, the model is loaded from its embedded representation, and the tensor arena is allocated. This initialization step prepares the inference engine before periodic execution begins.

2. **Input preparation:** For each inference cycle, the current input window is copied or formatted into the model input tensor. This step ensures that the data provided to the interpreter matches the expected tensor layout and quantized representation.

3. **Interpreter invocation:** The inference task calls the TFLM interpreter to execute the forward pass of the neural network. This stage represents the main computational workload of the task and is the primary focus of timing evaluation.

4. **Output generation:** After execution, the output tensor is read and interpreted as a six-class prediction. The predicted class or raw output values may then be passed to subsequent application logic or stored for analysis.

5. **Timing capture:** Runtime timestamps are taken around the inference operation so that execution behavior can be recorded. These measurements are used later to evaluate how inference timing is affected by concurrent task interference and scheduling conditions.

Together, these stages define the internal behavior of the inference task and provide a clear and reproducible inference pipeline for evaluating TinyML behavior under multitasking embedded conditions.

4.5 FreeRTOS Task Architecture

The system adopts a task-based organization in which the inference functionality and the auxiliary workload activities are implemented as separate FreeRTOS tasks. This structure allows the execution behavior of the system to be controlled at the scheduler level while preserving a clear separation between inference and sources of interference. The resulting architecture supports both realism and repeatability: the selected tasks reflect common embedded activities, yet their activation and computational demand remain configurable for experimental use.

The central task in the design is the **inference task**, which performs periodic TinyML inference using the deployed TFLM model. It is responsible for preparing the model input, invoking the interpreter, reading the output tensor, and recording timing information associated with each inference cycle. Because this task is the main object of study, its scheduling behavior is evaluated under different interference conditions created by the other workload tasks.

The **sensor workload task** represents sensing-related activity commonly found in embedded systems. Its role is to emulate the CPU demand and periodic behavior associated with sensor polling, signal acquisition, or lightweight preprocessing. Although simplified, this task introduces realistic short-interval activity that competes with inference for processor time.

The **communication workload task** emulates communication-related processing, such as protocol handling, packet preparation, or network stack interaction. In the system design, this task contributes additional computational load that reflects the communication overhead often present in connected embedded devices. Its inclusion helps model the effect of communication processing on inference timing under multitasking conditions.

The **I/O workload task** represents storage or peripheral-related activity, such as file operations or data transfer to nonvolatile memory. This task is included because embedded applications frequently interact with storage or peripheral interfaces whose execution can introduce additional delay and contention. By incorporating this type of activity into the design, the framework captures another important source of runtime interference.

If implemented separately, a **logging task** manages the recording or flushing of timing outputs and summary data. Separating logging from the main inference path helps reduce direct perturbation of the inference task while preserving structured runtime records for later analysis.

These tasks were chosen because they collectively represent a practical subset of embedded system behavior: sensing, computation, communication, storage, and measurement. At the same time, they are intentionally simplified so that the level of interference they impose remains controlled and repeatable. This makes the task architecture suitable for scheduling experiments in which interference patterns must be varied systematically without losing interpretability.

A summary of the task roles in the proposed system is provided in Table 4.2.

4.6 Scheduling and Interference Design

The proposed system adopts the FreeRTOS preemptive priority-based scheduling model, in which each task is assigned a fixed priority and the scheduler selects the highest-priority ready task for execution [9, 10]. Under this model, a newly released higher-priority task can preempt a lower-priority task, while tasks at the same priority level may share processor time according to the configured scheduling policy [9, 10]. This scheduling structure is appropriate for the present study because it makes the effect of task priorities explicit and allows scheduler-level interference to be

Table 4.2: Task Roles in the Proposed System

Task Name	**Functional Role**	**Periodic / Event-Driven Behavior**	**Resource Demand / Type of Load**	**Purpose in the Design**
Inference Task	Runs periodic TinyML inference using the deployed model	Periodic	Neural network inference, tensor handling, timing capture	Serves as the primary task under evaluation for latency and scheduling behavior
Sensor Workload Task	Simulates sensing-related activity	Periodic	Lightweight periodic CPU load representing polling or preprocessing	Introduces realistic sensing interference in the multitasking environment
Communication Workload Task	Emulates communication-related processing	Periodic or event-driven	CPU load associated with communication handling or protocol processing	Represents communication overhead common in connected embedded systems
I/O Workload Task	Emulates storage or peripheral-related activity	Periodic or event-driven	File, storage, or peripheral interaction load	Introduces interference associated with data movement and I/O activity
Logging Task	Records timing samples and summary outputs if implemented separately	Periodic or event-driven	Lightweight logging and output management load	Separates measurement recording from the critical inference path when needed

introduced in a controlled and repeatable manner.

The task set is organized using fixed-priority assignment. The inference task, together with the background workload tasks, is given a predefined priority for each experimental configuration. This approach keeps the scheduling policy itself unchanged while allowing the interference pattern to be modified through the relative priority relationships among tasks. In this way, the framework can expose how the timing behavior of inference changes when it is favored by the scheduler, when it competes with tasks of similar importance, or when it is placed below more aggressive background activity.

Although the ESP32-S3 provides a multicore platform, the scheduling analysis is intentionally structured around single-core execution. This design choice avoids the additional complexity

introduced by parallel task execution across cores and makes timing interactions easier to interpret. By confining the competing tasks to a single scheduling domain, the observed latency variation can be attributed more directly to preemption, context switching, release timing, and CPU contention rather than to cross-core effects. This is consistent with prior real-time analysis that emphasizes how scheduler and kernel overheads influence task response time in single-core embedded systems [10].

The interference design follows a simple principle: changing task priorities changes interference patterns. When the inference task has higher priority, it experiences fewer preemptions and can execute with relatively limited scheduler-induced delay. When background tasks are assigned equal or higher priority, they contend more strongly for processor time and can delay inference execution through preemption or postponed dispatch. In priority-driven systems, this type of interference is central to response-time behavior because each task must accommodate not only its own execution time but also preemption from higher-priority tasks and blocking effects introduced by lower-priority activities under resource contention [12, 11].

The background workload tasks are therefore not treated as incidental system activity, but as deliberate sources of controlled interference. Their role is to compete with the inference task for CPU time in a manner that resembles realistic embedded multitasking while remaining sufficiently structured for repeatable experiments. This design makes the framework sensitive to timing effects such as delayed dispatch, increased response time, and scheduler-induced variability, without yet introducing the experimental results themselves. The purpose at this stage is to establish the logic of the scheduling design: the system is intentionally arranged so that inference timing can be observed under different levels of scheduler pressure and task competition.

4.7 Background Workload Design

To evaluate inference behavior under realistic multitasking conditions, the system incorporates a set of background workload tasks that execute concurrently with the inference task. These workloads are designed to emulate three common classes of embedded activity: sensing, communication, and storage or peripheral I/O. Such functions are widely present in practical IoT

and embedded systems, where sensor nodes acquire data continuously, communication modules exchange messages over lightweight protocols, and local storage or peripheral interfaces support logging and state retention [3, 6, 4]. In the proposed framework, these activities are represented as simplified software tasks whose primary purpose is to introduce controlled execution demand and scheduler interference.

The **sensor workload** emulates the periodic behavior of sensing-related functions such as polling, data acquisition, and lightweight preprocessing. In many embedded systems, sensing tasks are activated at regular intervals and contribute short but repeated bursts of processor demand [3, 2]. The sensor workload in this design follows the same general pattern, producing periodic computation that competes with the inference task for CPU time without requiring a full physical sensing pipeline. This allows the framework to reflect the temporal presence of sensing activity while preserving experimental control.

The **communication workload** represents the processing cost associated with communication handling in connected embedded devices. This includes activities such as packet preparation, protocol stack interaction, and message transmission support. Communication is especially relevant in IoT-oriented systems, where lightweight publish-subscribe mechanisms such as MQTT are commonly used because of their suitability for constrained devices and low-latency data exchange [6, 4, 5]. In the present system, the communication workload is not intended to reproduce a complete networked application. Instead, it abstracts the computational demand and scheduler impact of communication-related processing so that its influence on inference timing can be observed in a repeatable way.

The **I/O workload** emulates storage-related or peripheral-related activity, including operations analogous to file access, logging support, or interaction with nonvolatile memory. Embedded applications frequently combine inference or control logic with some form of local data retention, whether for monitoring, debugging, or coordination purposes. The inclusion of an I/O-oriented workload therefore adds another realistic source of interference to the multitasking environment. Rather than implementing a full storage subsystem, the design uses a simplified abstraction that

captures the presence of additional execution demand and potential delay associated with output or persistence operations.

A central design objective of these workloads is **realistic interference with repeatability**. The tasks are realistic in the sense that they correspond to common embedded functions, but they remain simplified enough that their behavior can be reproduced across repeated experiments. This balance is important because the purpose of the workloads is not to model every implementation detail of a deployed application, but to create representative scheduler pressure around the inference task. In this way, the framework can expose timing sensitivity under conditions that resemble practical embedded operation while remaining experimentally interpretable.

A second design objective is **controllable CPU demand**. Each workload is structured so that its activation pattern and computational intensity can be adjusted in a predictable manner. This makes it possible to vary the amount of background interference without changing the overall system architecture. Similar workload-oriented evaluation logic has been used in prior embedded studies, where combinations of sleep, computation, and transmission behavior were used to generate controlled operating conditions for measurement and analysis [13]. In the present work, the same principle is adopted at the scheduler level: background tasks are constructed as configurable sources of interference so that the effect of contention on inference timing can be examined systematically.

Accordingly, the sensing, communication, and I/O workloads should be understood as **simplified abstractions of common embedded functions**, rather than as complete application implementations. Their role is to provide representative, configurable, and repeatable background activity around the inference task. This design choice keeps the system focused on the research objective of studying scheduler-level interference on TinyML inference timing, while avoiding unnecessary complexity that would reduce experimental clarity.

4.8 Timing Instrumentation and Logging Mechanism

The system incorporates a lightweight timing instrumentation and logging mechanism to capture runtime behavior during inference execution under multitasking conditions. Timing mea-

surements are obtained using the `esp_timer_get_time()` API, which provides microsecond-resolution timestamps suitable for embedded runtime profiling. The use of fine-grained timestamp capture is consistent with prior work emphasizing the importance of precise timing measurement in embedded and real-time systems [14]. In the proposed framework, timestamp collection is kept simple and localized so that the instrumentation remains compatible with the resource constraints of the ESP32-S3.

Timestamps are taken at selected points within the inference task. At minimum, the system records the start and end of each inference cycle so that the execution interval of a single invocation can be reconstructed from the raw measurements. Additional timestamps may also be placed around internal stages of the inference path, such as input preparation, interpreter invocation, and output handling, when finer-grained breakdown is required. This placement strategy allows the instrumentation to focus on the inference path itself while avoiding unnecessary measurement points in unrelated parts of the system.

The logging mechanism records the runtime values produced by this instrumentation in a structured form. For each inference cycle, the primary data captured are the timestamp-derived execution samples associated with that cycle. These raw samples form the basis of later offline analysis. In addition to the per-cycle measurements, the system generates compact summary information after an experimental run. Rather than storing only final aggregate results, the design preserves both the individual samples and the corresponding summaries so that execution traces can be examined in detail when needed. This approach is consistent with measurement-oriented embedded workflows that combine raw sample capture with structured post-run reporting [13].

Raw timing samples are stored on the device using SPIFFS, which provides a persistent file-system layer for embedded storage. This allows runtime measurements to be written locally without requiring continuous host-side monitoring during execution. Persistent storage is important in the present framework because experiments are performed over repeated inference cycles and under varying interference conditions, making it necessary to preserve the collected data for later retrieval and analysis. By writing timing records to SPIFFS, the system can retain both detailed

execution samples and summarized results across the full duration of an experimental run.

The summary generation process is performed after the relevant raw samples have been collected. The instrumentation module computes compact statistical summaries from the stored timing records and appends them to the output in a structured format. In this way, each experiment produces two complementary levels of data: a raw sample sequence for detailed inspection and a concise summary for direct comparison across runs. The purpose of this mechanism is not to interpret the timing behavior during execution, but to ensure that the data needed for later evaluation are captured consistently and preserved reliably.

To minimize distortion of the measured behavior, the logging path is kept lightweight and separated as much as possible from the critical inference execution path. The instrumentation is therefore designed to observe runtime behavior with limited additional software overhead, while still producing persistent and reproducible measurement records. This combination of microsecond-level timestamping, structured raw-sample collection, summary generation, and SPIFFS-based storage provides the data-capture foundation for the timing analysis presented in later chapters.

4.9 Memory and Resource Considerations

Memory and compute constraints are central to the design of the proposed system. Because the ESP32-S3 provides only embedded-scale resources, the inference framework must allocate memory carefully while keeping runtime overhead low enough to preserve meaningful timing behavior. These constraints influence not only the deployment of the TinyML model itself but also the structure of the workload tasks, the instrumentation strategy, and the logging mechanism [7, 8, 1].

A major design element is the **tensor arena allocation** used by TensorFlow Lite Micro. TFLM relies on a statically allocated memory region to store intermediate tensors, activations, and other interpreter-managed data during inference [7, 8]. In the proposed system, this approach is retained because static allocation is more predictable than dynamic allocation in a constrained embedded environment. The size of the tensor arena therefore becomes an important design

parameter: it must be large enough to support the deployed model and its intermediate buffers, but small enough to leave sufficient memory available for the operating system, workload tasks, stack usage, and logging support. This requirement directly affects model selection and reinforces the use of a lightweight quantized network architecture.

In addition to the tensor arena, the system must account for **heap and stack use** across all concurrent tasks. Each FreeRTOS task requires stack space, and the combined memory footprint of the inference task, background workloads, and optional logging support must remain within the limits of the platform. From a design perspective, this means that the task set cannot be expanded arbitrarily, and each task must remain intentionally compact. The software structure therefore favors simplified workload abstractions and lightweight runtime support rather than feature-rich implementations. This is consistent with embedded AI design practice, where model execution must coexist with the rest of the application inside a limited memory budget [1, 7].

A further consideration is **logging overhead**. Although timing instrumentation is necessary for evaluation, excessive logging can distort the very behavior being measured by consuming processor time, memory, and storage bandwidth. For this reason, the proposed system keeps the logging path lightweight and records only the timing data needed for later analysis. Raw samples are preserved, but the logging mechanism is structured to avoid unnecessary processing in the critical inference path. This design choice reflects the broader principle that measurement support in embedded systems should remain accurate while introducing as little interference as possible [13, 14]. The same consideration also motivates the use of persistent on-device storage through SPIFFS, which allows runtime data to be retained without continuous host interaction.

Taken together, these constraints illustrate the broader **requirements of embedded deployment**. The system cannot assume abundant RAM, unrestricted storage, or negligible scheduler overhead. Instead, memory allocation, task structure, model complexity, and measurement support must all be designed with resource limitations in mind. These constraints are not incidental implementation details; they shape the architecture of the framework itself. In particular, they motivate the use of an INT8 quantized model, a statically allocated tensor arena, a compact set of workload

tasks, and a lightweight instrumentation strategy. As a result, the final system remains suitable for deployment on a realistic embedded target while still supporting controlled evaluation of TinyML inference under multitasking conditions.

4.10 Chapter Summary

This chapter presented the design of the embedded TinyML inference framework developed for this study. It described the ESP32-S3 target platform and the supporting software environment built on ESP-IDF, FreeRTOS, TensorFlow Lite Micro, `esp-nn`, and SPIFFS-based logging support. The chapter then outlined the overall system architecture, including the inference engine, scheduler-managed task structure, background workload modules, timing instrumentation, and logging path. In addition, it detailed the deployed HAR-based model and its inference pipeline, the organization of the FreeRTOS task set, the scheduling and interference design, the structure of the background workloads, and the mechanisms used to capture and store runtime timing data. Memory and resource constraints were also discussed to show how embedded deployment limits influenced model selection, task design, and instrumentation choices.

Taken together, these design elements establish a complete and controlled framework for studying the timing behavior of TinyML inference under multitasking embedded conditions. Building on this system design, Chapter 5 presents the experimental setup, including the hardware and software configuration, task parameters, scheduling scenarios, and evaluation procedure used in the empirical study.

Chapter 5

Experimental Setup

This chapter presents the experimental setup used to evaluate the timing behavior of the deployed TinyML inference task under controlled multitasking conditions. Its purpose is to define the practical testbed, fixed execution environment, evaluated workload scenarios, and measurement workflow used throughout the study. In particular, the chapter describes the hardware and software platform, the deployed inference configuration, the design of concurrent background workloads, the scheduling scenarios considered, and the procedure used to record and process runtime timing data. By establishing these elements in a systematic and reproducible manner, this chapter provides the basis for the performance results and comparative analysis presented in the following chapter.

5.1 Experimental Objectives

The experiments in this study are designed to evaluate how multitasking and scheduler configuration influence the temporal behavior of embedded TinyML inference. First, the experiments examine the effect of concurrent background workload on inference timing by observing how sensing, communication, and I/O activity alter the execution behavior of the inference task relative to low-interference operation. Second, the experiments investigate the effect of task-priority assignment on schedulability by comparing configurations in which the inference task is given favorable, competing, or disadvantaged priority levels with respect to the other system tasks. Third, the experiments assess how increasing levels of workload contention affect timing predictability, with particular attention to changes in execution variability, deadline satisfaction, and the emergence of unstable or starvation-prone operating conditions. Together, these objectives provide a structured basis for analyzing the timing robustness of TinyML inference in a realistic multitasking embedded environment.

5.2 Testbed and Execution Environment

The experiments in this study are conducted using a fixed hardware and software configuration so that differences in timing behavior can be attributed to the evaluated workload and scheduling conditions rather than to changes in the execution platform. The complete system design, including the target platform, software stack, inference framework, and logging architecture, has already been described in Chapter 4. Accordingly, this section summarizes only the execution conditions that remain fixed across all experiments and that are directly relevant to the interpretation of the results.

5.2.1 Hardware Platform

All experiments are performed on the same ESP32-S3-based embedded platform. The use of a single hardware target ensures consistency across runs and allows the observed timing variations to be interpreted as consequences of workload interference and scheduler behavior rather than hardware differences.

5.2.2 Software Stack

The execution environment is based on the ESP-IDF framework with FreeRTOS as the underlying real-time operating system. The deployed inference application uses TensorFlow Lite Micro for on-device model execution. Additional implementation details regarding the software stack and its role in the overall system architecture are provided in Chapter 4.

5.2.3 Core Allocation and Execution Constraints

Although the ESP32-S3 supports multicore execution, all experimental tasks are pinned to a single core throughout the evaluation. This constraint is imposed to create a controlled contention environment in which the inference task and background workload tasks compete for the same processing resource. As a result, the measured timing behavior reflects scheduler-level interference more directly and can be compared consistently across scenarios.

5.2.4 Logging and Storage Environment

Runtime observation is supported through serial monitoring together with SPIFFS-based persistent logging. Serial output is used to monitor execution progress during each run, while timing data is stored on-device for subsequent analysis. The detailed design of the timing and logging mechanism has already been presented in Chapter 4; in the context of this chapter, it is sufficient to note that the same measurement and storage method is used in all experimental scenarios.

5.3 Deployed Model and Inference Configuration

The inference workload used throughout the experiments is kept fixed so that the observed timing differences across scenarios can be attributed to scheduling and interference effects rather than to changes in model structure or runtime configuration. The deployed model is a quantized embedded machine learning model executed on-device through TensorFlow Lite Micro (TFLM), which provides the inference framework for resource-constrained microcontroller platforms [7]. In the context of this study, the model serves as the primary real-time workload of interest, representing a practical TinyML inference task that must execute periodically while sharing processor time with concurrent background activities.

More specifically, the deployed model performs human activity recognition based on the UCI Human Activity Recognition dataset [15]. The model is included in the system as a representative embedded classification task and is invoked repeatedly as part of the periodic inference thread. Its role in the experimental system is not to evaluate classification accuracy, but to provide a realistic and repeatable TinyML workload whose execution timing can be observed under different multitasking and scheduling conditions.

The inference task is configured with a fixed activation period of 10 ms and a matching relative deadline of 10 ms for all experiments. This configuration causes each invocation to be treated as a periodic real-time workload that is expected to complete within the same interval in which it is released. Keeping the period and deadline identical across all scenarios ensures that the

schedulability of the inference task can be compared directly under varying levels of interference and priority assignment.

Each experimental run is executed for 1000 consecutive inference activations. This fixed run length provides a sufficiently large set of runtime samples for characterizing timing behavior while maintaining consistency across scenarios. The same deployed model, input structure, invocation path, and runtime configuration are used in every case so that the only intentional changes between experiments arise from the surrounding workload and scheduling setup.

In addition to the fixed period and deadline, the inference configuration remains constant in several other respects throughout the study. The same quantized model file is used in all runs, inference is executed through the same TFLM interpreter path, and the memory resources allocated for model execution remain unchanged across scenarios. This fixed configuration is important for experimental control because it isolates the scheduler-level effects under investigation and avoids confounding the timing results with changes in model size, interpreter behavior, or memory allocation.

5.4 Background Workload Design

To evaluate inference timing under realistic multitasking conditions, the experimental system includes a set of controlled background tasks that execute concurrently with the periodic TinyML inference task. These background activities are designed to emulate common sources of embedded workload interference while remaining simple enough to be reproduced consistently across repeated runs. In this study, interference is introduced through three categories of concurrent activity: a sensor-oriented task, a communication-oriented task, and an I/O-oriented task. Together, these tasks create a practical but controlled execution environment in which the inference task must compete for processor time and system resources.

Rather than relying on external peripherals, live network traffic, or uncontrolled input streams, the workloads are implemented as synthetic task activities with fixed activation patterns and predefined computational demand. This approach allows the experiments to preserve repeatability

while still capturing the essential forms of interference that arise in embedded IoT applications. Each background task is therefore intended to represent a functional class of system activity rather than a full end-to-end application service.

5.4.1 Sensor Task Design

The sensor workload is designed to emulate periodic sensing and lightweight preprocessing activity. It executes as a regularly released background task that consumes processor time through a controlled computation loop. Conceptually, this task represents the type of short, recurring work commonly associated with embedded data acquisition, such as polling a sensor, performing basic filtering, or preparing a sample for later use.

The sensor task is configured with a relatively short activation period so that it introduces frequent interference into the schedule. This makes it an appropriate source of repeated preemption pressure on the inference task, especially in scenarios where the sensor task is assigned equal or higher priority. Because the workload is generated using a fixed software loop rather than live sensor input, its execution demand remains stable across experimental runs.

5.4.2 Communication Task Design

The communication workload is intended to represent the processing cost associated with embedded data transmission or protocol handling. In the experimental implementation, this task performs controlled communication-related computation to introduce a heavier and more sustained processing burden than the sensor task. It therefore models the type of background work that might arise from packet preparation, message handling, cryptographic processing, or communication stack activity in a connected embedded system.

Compared with the sensor task, the communication task contributes a larger amount of processor demand per activation and is used to create meaningful scheduling contention during the experiments. When assigned elevated priority, it becomes an important source of interference that can delay inference execution and increase timing variability. Its use as a software-defined workload

also ensures that the same amount of activity is reproduced consistently in each evaluated scenario.

5.4.3 I/O Task Design

The I/O workload is designed to emulate periodic storage-related activity. This task represents the type of background work associated with file-system interaction, buffered writes, or other data persistence operations that often appear in embedded systems performing logging or event recording. In the present setup, the I/O task is activated periodically and introduces additional execution demand that is distinct from the CPU-oriented behavior of the sensing and communication tasks.

Although the I/O task is not intended to model the full complexity of external storage subsystems, it provides a useful approximation of recurring storage interference within the software environment used in this study. Its inclusion broadens the range of workload types competing with the inference task and helps create a more representative multitasking execution context.

5.4.4 Rationale for Controlled Interference

The background workloads used in this study are intentionally synthetic but repeatable. This design choice is important for experimental control. If interference were generated through uncontrolled external inputs, live communication traffic, or irregular peripheral behavior, then observed timing differences could reflect environmental variation rather than the scheduler-level effects under investigation. By contrast, software-defined workloads with fixed periods and stable execution demand make it possible to reproduce the same operating conditions across multiple runs and compare scenarios more directly.

At the same time, the workloads are chosen to reflect realistic categories of embedded activity. Sensor handling, communication processing, and storage operations are common functions in practical TinyML-enabled IoT systems, and each can compete with inference for processor time in different ways. The workload design therefore balances realism with repeatability: it does not attempt to reproduce every low-level detail of a complete deployed application, but it does provide

representative and controlled interference sources that are well suited to schedulability and timing analysis.

5.5 Experimental Variables and Controlled Factors

The experimental design distinguishes between variables that are intentionally changed and factors that are kept constant across all runs. This separation is necessary to ensure that the observed timing behavior can be attributed primarily to workload interference and scheduling configuration. In this study, the independent variables define the experimental conditions, the dependent variables define the measured timing outcomes, and the controlled factors define the fixed configuration used to preserve comparability across scenarios.

5.5.1 Independent Variables

The independent variables in this study are the scenario type, task-priority assignment, and contention level. The scenario type determines the overall operating condition being evaluated, such as baseline execution, periodic interference, or priority-based scheduling contention. The task-priority assignment defines the relative scheduling relationship between the inference task and the background workload tasks. By changing the priority of the inference task relative to the sensor, communication, and I/O tasks, the experiments evaluate how favorable or unfavorable scheduling positions affect inference execution.

The contention level represents the degree of competition experienced by the inference task during execution. Lower-contention scenarios provide limited interference, while higher-contention scenarios place the inference task under stronger competition from background workload tasks. These independent variables are varied in a controlled manner to examine how scheduler-level interference affects inference timing and schedulability.

5.5.2 Dependent Variables

The dependent variables are the timing measurements collected from each experimental run. These include inference latency, jitter, observed worst-case execution time (WCET), deadline miss ratio, and lateness. These metrics are used to characterize how the inference task behaves under each scheduling and workload condition.

Since the conceptual definitions of these timing metrics were already presented in Chapter 2, they are not redefined in detail here. Instead, this chapter focuses on how the metrics are produced from the experimental runs and used to compare the evaluated scenarios.

5.5.3 Controlled Factors

Several factors are kept constant across all experiments to ensure a fair comparison between scenarios. The same ESP32-S3 hardware platform is used throughout the study, and the firmware environment is kept fixed. The deployed TinyML model, inference configuration, and runtime path are also unchanged across all runs. In addition, all tasks are pinned to the same processing core to maintain a consistent single-core contention environment.

The logging method and measurement procedure are also held constant. Timing data is collected using the same instrumentation approach and stored using the same logging mechanism in each experiment. Finally, each run uses the same number of inference activations. By controlling these factors, the evaluation isolates the effects of scenario design, task priority, and workload contention on inference timing behavior.

Table 5.1: Experimental Variables and Controlled Factors

Category	Factors
Independent variables	Scenario type, task-priority assignment, contention level
Dependent variables	Latency, jitter, observed WCET, deadline miss ratio, lateness
Controlled factors	Hardware platform, firmware environment, deployed model, core placement, logging method, activation count

5.6 Task Parameters Used in the Experiments

The task set used in the experiments consists of one periodic inference task and three background workload tasks. The inference task represents the primary TinyML workload being evaluated, while the sensor, communication, and I/O tasks introduce controlled multitasking interference. Each task is assigned a fixed period, workload description, deadline, and activation pattern so that the experiments can be repeated consistently across scheduling scenarios.

Table 5.2 summarizes the task parameters used throughout the evaluation. These parameters remain fixed across the evaluated priority configurations; only the relative task priorities are changed between scheduling scenarios.

Table 5.2: Task Parameters Used in the Experiments

Task	Functional Role	Period	Execution Demand Workload Description	Deadline	Activation Pattern
Inference	Executes the deployed TinyML model	10 ms	One TensorFlow Lite Micro inference per activation	10 ms	Periodic
Sensor	Emulates sensing and lightweight preprocessing activity	10 ms	50,000-iteration CPU workload loop	10 ms	Periodic
Communication	Emulates communication related background processing	20 ms	80,000-iteration CPU workload loop	20 ms	Periodic
I/O	Emulates storage or file system related background activity	50 ms	30,000-iteration CPU workload loop	50 ms	Periodic

5.7 Evaluated Scheduling Scenarios

The experiments are organized into a set of scheduling scenarios that progressively increase the level of interference experienced by the inference task. These scenarios are designed to separate the timing behavior of the inference workload under low-interference operation, general periodic

background activity, and explicit priority-based contention. The baseline and periodic interference scenarios establish reference behavior, while the Priority-A, Priority-B, and Priority-C scenarios evaluate how task-priority assignment affects schedulability under competing workload conditions.

5.7.1 Baseline Scenario

The baseline scenario evaluates the inference task under minimal interference. In this case, the inference workload is executed without the full set of competing background tasks. This scenario provides a reference measurement for the deployed model and runtime environment when scheduler contention is limited. The baseline results are used as a comparison point for determining how much additional delay and variability are introduced when background workloads are added.

5.7.2 Periodic Interference Scenario

The periodic interference scenario introduces concurrent background activity while keeping the experimental setup controlled and repeatable. In this case, periodic workload tasks execute alongside the inference task to emulate the multitasking behavior of an embedded IoT system. This scenario helps characterize how inference timing changes when the system is no longer dedicated to inference alone but must also accommodate sensing, communication, or I/O-related activity.

Unlike the later priority-based scenarios, the purpose of this scenario is not primarily to test a specific priority configuration. Instead, it serves as an intermediate condition between baseline execution and stronger scheduling contention. It shows the effect of adding periodic system activity before evaluating the more aggressive priority assignments used in the remaining scenarios.

5.7.3 Priority-A Scenario

The Priority-A scenario represents the most favorable scheduling condition for the inference task among the priority-based configurations. In this scenario, the inference task is assigned a higher priority than the background workload tasks. As a result, the inference task is protected from most background interference and can preempt lower-priority workload activity when it becomes ready

to execute.

This configuration is intended to approximate a scheduling-aware design in which the real-time inference workload is treated as the most timing-sensitive task in the system. The Priority-A scenario is therefore used to evaluate whether the inference task can maintain stable timing behavior and meet its deadline when it is given scheduling preference over competing system activity.

5.7.4 Priority-B Scenario

The Priority-B scenario represents a contended execution condition. In this configuration, selected background workload tasks are assigned a higher priority than the inference task, allowing them to preempt or delay inference execution. This creates a more stressful multitasking environment than Priority-A and is intended to show how inference timing behaves when the model execution is no longer the dominant scheduling priority.

This scenario is important because it reflects a common embedded-system risk: inference may be integrated into an existing application where sensing, communication, or processing tasks are already assigned significant scheduling importance. Under this condition, the inference task may experience increased latency, greater timing variability, and a higher probability of deadline misses.

5.7.5 Priority-C Scenario

The Priority-C scenario represents an extreme contention case and is used to examine starvation-prone behavior. In this configuration, the inference task is assigned a lower priority than the competing background workload tasks. Since the higher-priority background tasks can repeatedly execute before the inference task receives sufficient processor time, this scenario may prevent inference from making reliable forward progress.

The purpose of Priority-C is not to represent a recommended deployment configuration. Instead, it serves as a stress case that illustrates the consequence of assigning a timing-sensitive TinyML inference task below sustained high-priority background activity. If the inference task

cannot complete consistently under this condition, the result provides qualitative evidence that inappropriate priority assignment can make an otherwise functional inference workload unschedulable in a multitasking embedded system.

5.8 Task Priorities for Evaluated Scheduling Scenarios

The evaluated scenarios differ primarily in the relative priority assigned to the inference task and the background workload tasks. Table 5.3 summarizes the task-priority assignments used in the priority-based experiments. In FreeRTOS, a larger priority value represents a higher scheduling priority. Therefore, the table shows whether the inference task is protected from background interference, placed in contention with other tasks, or placed below competing workload activity.

The baseline and periodic interference cases are included as reference scenarios. In these cases, the priority relationship is not the main experimental variable. The Priority-A, Priority-B, and Priority-C configurations are the main scheduling scenarios used to evaluate the effect of task-priority assignment on inference schedulability.

Table 5.3: Task Priorities for Evaluated Scheduling Scenarios

Scenario	Inference	Sensor	Communication	I/O	Remarks
Baseline	-	-	-	-	Inference reference case with minimal competing workload.
Periodic	-	-	-	-	Reference case with periodic background activity; priority assignment is not the main variable.
A	5	3	3	2	High-priority inference configuration; represents favorable scheduling for inference.
B	3	4	4	2	Contended execution configuration; selected background tasks can preempt inference.
C	2	5	5	5	Starvation-oriented stress case; inference is placed below sustained background work

5.9 Measurement and Logging Procedure

Each experimental run follows the same measurement procedure to ensure consistency across scenarios. Before a run begins, the firmware is built and flashed to the ESP32-S3 with the selected experimental configuration. The scenario configuration determines which tasks are enabled

and what priority assignments are applied. After the firmware starts, the system initializes the inference runtime, mounts the logging storage, and creates the inference and background workload tasks according to the selected scenario.

Once execution begins, the inference task is released periodically according to the fixed inference period. For each activation, timestamps are recorded around the inference execution path so that the duration of the inference operation can be measured. These timestamps provide the raw timing information needed to compute the latency of each activation and to determine whether the corresponding deadline is satisfied. The same timestamping method is used across all scenarios so that the measurements remain comparable.

During each run, the system records timing samples and stores them in the on-device logging storage. Runtime progress may also be observed through the serial monitor, which is used to confirm that the experiment is executing as expected. After the required number of inference activations has been completed, the recorded samples are preserved as raw timing logs. These logs are then extracted for offline analysis, where the timing metrics and plots used in the results chapter are generated.

The overall measurement procedure can be summarized as follows:

1. Build and flash the firmware for the selected experimental scenario.
2. Start the system and initialize the inference runtime, task set, and logging mechanism.
3. Configure the task priorities and workload behavior associated with the selected scenario.
4. Execute the inference task and any enabled background workload tasks.
5. Record timestamps for each inference activation.
6. Store the raw timing samples using the on-device logging mechanism.
7. Save or extract the generated logs for offline processing and analysis.

This procedure is repeated for each evaluated scenario. By keeping the measurement method fixed, the experimental setup ensures that differences in the recorded timing behavior are caused by the scenario configuration rather than by changes in data collection or logging practice.

5.10 Data Processing and Analysis Workflow

After each experimental run, the raw timing logs are extracted and processed offline to produce the final measurements used in the results chapter. Each recorded sample corresponds to one inference activation and contains the timing information needed to characterize the runtime behavior of the inference task. The first step in the analysis workflow is to extract the per-activation inference latency from the recorded timestamps. This latency value represents the measured completion time of each inference activation under the selected scheduling scenario.

After latency values are extracted, each activation is compared against the fixed 10 ms deadline. Any activation whose latency exceeds the deadline is counted as a deadline miss. The deadline miss ratio is then computed by dividing the number of missed activations by the total number of inference activations in the run. For missed activations, lateness is also computed as the amount of time by which the measured latency exceeds the deadline.

The observed worst-case execution time is obtained by selecting the maximum measured inference latency within a run. This value is reported as an observed WCET because it is based on empirical measurement rather than formal static timing analysis. Jitter and other summary statistics are then computed from the full set of latency samples to describe the variability of inference timing under each scenario.

Finally, the processed data is organized into summary tables and plots. Summary tables are used to compare key timing values across scenarios, including mean latency, observed worst-case latency, deadline misses, and lateness. Plots are used to visualize timing distributions and highlight differences in predictability between scenarios. This workflow transforms raw runtime logs into the quantitative results presented and discussed in Chapter 6.

5.11 Validity Considerations and Limitations

Several limitations should be considered when interpreting the experimental results. First, the evaluation is conducted on a single hardware platform. Although the ESP32-S3 is a realistic microcontroller-class platform for TinyML deployment, the measured timing behavior may differ on other processors, boards, or embedded architectures. Therefore, the results should be interpreted as representative of the selected platform rather than as universal timing guarantees for all TinyML systems.

Second, the experiments use one deployed model and one application type. The selected model provides a repeatable embedded inference workload for evaluating scheduling behavior, but different model architectures, input sizes, operator types, or memory requirements may produce different timing characteristics. As a result, the findings describe the scheduling sensitivity of the evaluated inference workload rather than all possible TinyML applications.

Third, the background workloads are controlled and synthetic. This design improves repeatability and allows the effect of scheduling interference to be isolated more clearly. However, synthetic workloads do not capture every detail of full real-world sensing, communication, or storage subsystems. The results therefore emphasize controlled scheduler-level interference rather than complete application-level behavior under uncontrolled deployment conditions.

Fourth, the reported worst-case execution time is an observed value rather than a formally derived WCET. The maximum latency measured during an experimental run provides useful evidence of the worst observed behavior under the tested conditions, but it does not prove an absolute upper bound for all possible executions. For this reason, the term observed WCET is used throughout the evaluation.

Finally, the Priority-C scenario is treated as an extreme starvation-oriented case. If the system does not complete the full run under this configuration, the result is interpreted qualitatively rather than as a complete statistical comparison. In that case, repeated failure to make forward progress or watchdog-triggered behavior is used as evidence that the selected priority assignment

can make the inference task effectively unschedulable under sustained high-priority background activity.

5.12 Chapter Summary

This chapter defined the experimental setup used to evaluate TinyML inference timing under controlled multitasking conditions. It described the fixed testbed and execution environment, the deployed model and inference configuration, the design of background workload tasks, the experimental variables, the task parameters, and the evaluated scheduling scenarios. It also presented the measurement procedure and the workflow used to process raw timing logs into final latency, deadline, and variability measurements.

By establishing a consistent and reproducible experimental framework, this chapter provides the foundation for the results presented in the next chapter. Chapter 6 uses the measurement outputs defined here to analyze how workload contention and task-priority assignment affect inference latency, deadline satisfaction, and timing predictability.

Chapter 6

Results and Analysis

This chapter presents and analyzes the experimental results obtained from the scheduling configurations described in Chapter 5. The purpose of the evaluation is to determine how FreeRTOS task-priority assignment and background workload interference affect the timing behavior of TinyML inference on a resource-constrained embedded platform. The analysis focuses on whether the inference task can maintain its intended 10 ms periodic execution requirement while coexisting with representative sensing, communication, and storage workloads.

The experiments are organized around three scheduling configurations. Priority-A represents a favorable configuration in which the inference task is assigned higher priority than the background tasks. Priority-B increases the priority of competing background workloads to evaluate the effect of stronger scheduling interference. Priority-C represents an extreme stress configuration in which the inference task is assigned lower priority than all major background activities. Together, these configurations provide a progression from near-schedulable execution to severe timing degradation and finally to starvation.

The results show that the deployed TinyML model is computationally feasible on the ESP32-S3 platform when considered in isolation or under favorable scheduling conditions. However, the same inference workload exhibits substantially different timing behavior depending on task-priority assignment. This indicates that model size and average inference cost alone are insufficient for evaluating embedded TinyML readiness. Instead, real-time performance must be evaluated at the system level, including scheduler behavior, task interference, and background workload composition.

6.1 Priority-A Scheduling Configuration

In the Priority-A experiment, the inference task was assigned a higher priority than the background sensor, communication, and I/O tasks. This configuration was designed to evaluate whether favorable scheduling can preserve real-time inference performance under concurrent embedded workload conditions. The experiment was conducted over 1000 inference activations, with each activation subject to a 10 ms deadline.

Table 6.1: Timing Results for Priority-A Configuration

Metric	**Value**
Mean latency	8248.95 μs
Standard deviation	1175.20 μs
Minimum latency	8074 μs
Maximum latency (observed WCET)	23289 μs
Deadline	10000 μs
Deadline misses	15 / 1000 (1.50%)
Worst-case lateness	13290 μs

The timing results for Priority-A are summarized in Table 6.1. The inference task achieved a mean latency of 8248.95 μs, which remains below the 10 ms deadline. This result indicates that, under favorable scheduling conditions, the deployed TinyML workload can usually complete within the required timing window. The standard deviation of 1175.20 μs is relatively moderate compared with the mean, suggesting that most inference executions remain clustered around the nominal execution range of approximately 8 ms.

The minimum observed latency was 8074 μs, which is close to the mean value. This narrow separation between the minimum and mean latency suggests that the core inference workload itself is relatively stable when it is allowed to execute with limited preemption. This behavior is expected for a statically deployed TensorFlow Lite Micro inference model because memory allocation is performed ahead of execution through the tensor arena, and repeated invocations follow the same model execution path [7, 8].

However, the maximum observed latency was 23289 μs, which exceeds the 10 ms deadline by a significant margin. As a result, 15 out of 1000 activations missed their deadline, corresponding

to a deadline miss rate of 1.50%. The worst-case lateness was 13290 μs. These outliers show that even when inference is assigned the highest application-level priority among the evaluated tasks, occasional long-latency events can still occur due to system-level interference, kernel overhead, interrupt activity, logging behavior, or short periods of background execution.

From a real-time systems perspective, Priority-A can therefore be described as near-schedulable rather than strictly schedulable. The average behavior is acceptable, and most activations complete within the deadline, but the existence of deadline misses prevents the configuration from providing a hard real-time guarantee. This distinction is important because embedded TinyML systems are often evaluated using average latency alone, while deadline-constrained systems must also account for worst-case and tail-latency behavior.

6.2 Priority-B Scheduling Configuration

In the Priority-B experiment, the priorities of competing background tasks were increased relative to the inference task. This configuration was designed to intensify scheduling contention and evaluate how sensitive TinyML inference timing is to interference from sensing, communication, and I/O workloads. Unlike Priority-A, where inference receives favorable scheduling treatment, Priority-B places inference in a more contested execution environment.

Table 6.2: Timing Results for Priority-B Configuration

Metric	**Value**
Mean latency	38363.46 μs
Standard deviation	41886.39 μs
Minimum latency	7527 μs
Maximum latency (observed WCET)	350920 μs
Deadline	10000 μs
Deadline misses	590 / 1000 (59.00%)
Worst-case lateness	340921 μs

The timing results for Priority-B are summarized in Table 6.2. Under increased scheduling contention, the mean latency rises sharply to 38363.46 μs. This value is nearly four times the specified 10 ms deadline, indicating that the inference task is no longer able to maintain its intended

periodic behavior under this priority assignment.

The standard deviation also increases substantially to 41886.39 μs. This large variability indicates that inference execution is no longer clustered around a stable nominal value. Instead, the task experiences a wide range of response times depending on when it becomes ready, what background tasks are executing, and how long those tasks occupy the processor before the inference task can resume. The minimum latency of 7527 μs shows that the inference workload can still complete quickly when it obtains sufficient processor time. However, the maximum observed latency of 350920 μs demonstrates that some activations are delayed by hundreds of milliseconds.

The deadline miss rate under Priority-B is 59.00%, with 590 missed deadlines out of 1000 activations. This means that more than half of all inference activations fail to complete within the intended 10 ms timing window. The worst-case lateness reaches 340921 μs, showing that some activations complete extremely late relative to the deadline. These results indicate a clear loss of real-time schedulability.

The contrast between the low minimum latency and the very large maximum latency is especially important. It shows that the model itself is not always computationally too expensive for the platform. Instead, the primary issue is the scheduling environment in which the model executes. When background activities are allowed to preempt or delay inference more aggressively, the same model transitions from near-schedulable behavior to highly unpredictable timing behavior. This supports the central argument of this thesis: TinyML inference timing must be analyzed as part of a complete multitasking embedded system, not only as an isolated model benchmark.

6.3 Comparative Timing Analysis

A direct comparison between Priority-A and Priority-B demonstrates the strong influence of task-priority assignment on inference timing behavior. Both experiments use the same deployed inference workload, the same deadline, and the same target platform. The primary difference is the scheduling relationship between the inference task and the competing background tasks. Therefore, the observed timing degradation can be attributed primarily to scheduler-level contention rather

than to changes in the neural network model.

The 10 ms deadline is selected to match the periodic activation interval of the inference task. In the implemented system, inference is intended to process new sensor-derived input at a fixed 10 ms interval. This value represents a practical real-time requirement for time-sensitive embedded sensing applications. Although the deadline could be varied in future experiments, it is kept fixed in this study to isolate the effects of scheduling configuration and workload contention on timing performance.

Table 6.3: Comparison of Priority-A and Priority-B Timing Results

Metric	**Priority-A**	**Priority-B**
Mean latency	8248.95 μs	38363.46 μs
Standard deviation	1175.20 μs	41886.39 μs
Minimum latency	8074 μs	7527 μs
Maximum latency	23289 μs	350920 μs
Deadline misses	15 / 1000	590 / 1000
Deadline miss rate	1.50%	59.00%
Worst-case lateness	13290 μs	340921 μs

Table 6.3 summarizes the difference between the two measurable scheduling configurations. Under Priority-A, the inference task remains close to the required timing target. The mean latency is below the deadline, and only a small fraction of activations miss the deadline. Under Priority-B, the mean latency exceeds the deadline by a large margin, and the deadline miss rate increases from 1.50% to 59.00%.

The increase in mean latency from 8248.95 μs to 38363.46 μs represents more than a fourfold degradation in average response time. The standard deviation increases even more dramatically, from 1175.20 μs to 41886.39 μs. This indicates that Priority-B does not merely shift the latency distribution upward; it also makes the system significantly less predictable. Predictability is a central requirement in real-time systems because a system with moderate average performance but extreme timing variability may still fail to satisfy deadline requirements.

Figure 6.1 further illustrates this difference using the empirical cumulative distribution of inference latency. The Priority-A curve rises steeply within a narrow latency interval, indicating that

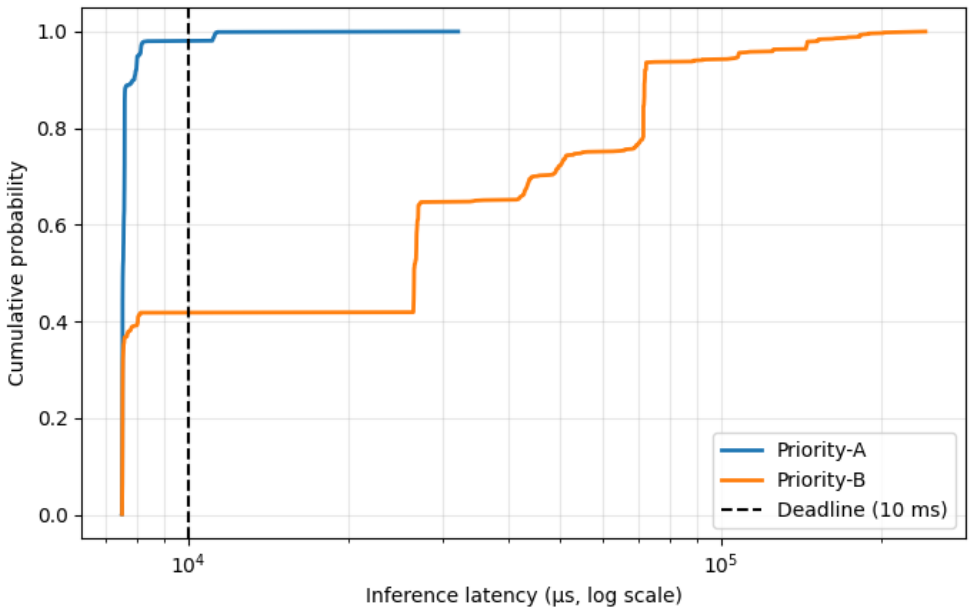


Figure 6.1: Empirical cumulative distribution of inference latency under Priority-A and Priority-B scheduling. Priority-A remains tightly concentrated near the nominal execution range, whereas Priority-B shows substantially greater dispersion and heavier tail behavior under increased contention.

most activations complete within a relatively compact range near the nominal inference time. In contrast, the Priority-B curve spreads across a much wider latency interval. This long-tail behavior indicates that many activations experience substantial delay before completion.

The ECDF representation is useful because it shows more than the average case. In deadline-constrained systems, the tail of the latency distribution is often more important than the mean. A system may appear acceptable based on average latency while still producing occasional deadline violations that are unacceptable in real-time operation. The Priority-B distribution demonstrates this issue clearly: even though some activations complete within the nominal range, a large portion of the distribution extends beyond the 10 ms deadline.

These results reveal a schedulability boundary between the two configurations. Priority-A operates in a near-schedulable region, where most inference activations complete on time but occasional outliers remain. Priority-B pushes the system into an unschedulable regime, where deadline misses become frequent, and timing behavior becomes highly variable. This transition

occurs without changing the deployed model, showing that the scheduling policy can determine whether the same TinyML workload is viable or non-viable in a real-time embedded system.

This behavior is consistent with established real-time scheduling theory. In fixed-priority preemptive systems, response time depends not only on a task's own execution time but also on interference from higher-priority tasks, blocking effects, and operating system overhead [11, 12, 10]. Although this experiment does not focus specifically on mutex-based priority inversion, it demonstrates the broader issue that lower-priority inference can be delayed substantially when higher-priority activities consume processor time. Prior formal analysis of FreeRTOS also shows that multitasking behavior can produce starvation and subtle scheduling failures that are difficult to expose through simple testing alone [9].

Therefore, the comparison between Priority-A and Priority-B supports one of the main findings of this thesis: real-time correctness in embedded TinyML systems depends on both model-level efficiency and system-level scheduling discipline. A lightweight model may still fail to meet deadlines if it is integrated into a poorly prioritized multitasking environment.

6.4 Deadline Miss and Lateness Analysis

The deadline miss results provide a direct measure of whether the inference task satisfies its periodic timing requirement. Under Priority-A, the system records 15 deadline misses out of 1000 activations. Under Priority-B, the system records 590 deadline misses out of 1000 activations. This change represents a shift from occasional deadline violation to frequent deadline failure.

The worst-case lateness values further emphasize this transition. Priority-A has a worst-case lateness of 13290 μs, meaning that the most delayed activation completed approximately 13.29 ms after its deadline. Priority-B has a worst-case lateness of 340921 μs, meaning that the most delayed activation completed approximately 340.92 ms after its deadline. This difference is significant because lateness reflects not only whether a deadline was missed, but also the severity of the miss.

In soft real-time applications, occasional small deadline misses may be tolerable depending on the application domain. For example, a sensing application may tolerate rare late classifications

if the system recovers quickly and continues processing future samples. However, large lateness values can cause stale inference results, missed events, unstable control behavior, or accumulation of delayed work. In Priority-B, the magnitude of worst-case lateness indicates that the inference task is not merely slightly late; it is delayed long enough to undermine the usefulness of periodic inference.

The deadline miss behavior also shows why average latency alone is insufficient. Priority-A and Priority-B differ not only in their means, but also in how often and how severely they violate the deadline. For embedded TinyML systems used in monitoring, alerting, or control, the relevant question is not simply whether the model can run on the microcontroller. The more important question is whether the model can run at the required time while the rest of the embedded system is active.

6.5 Priority-C Starvation Behavior

The Priority-C configuration provides qualitative evidence of starvation under extreme scheduling interference. In this setup, the inference task was assigned a lower priority than all competing background activities, including sensing, communication, and I/O-related workloads. As a result, the inference task was repeatedly preempted by higher-priority tasks and was unable to make sufficient forward progress to complete the experiment.

Unlike Priority-A and Priority-B, Priority-C did not produce a complete set of valid latency samples. Therefore, it should not be interpreted as an omitted quantitative result. Instead, it represents an experimentally observed failure mode. The inability to complete the target sample count is itself a meaningful result because it shows that the system crossed from degraded timing behavior into a starvation condition.

Table 6.4 summarizes the observed behavior under Priority-C. The execution logs indicate that processor time was frequently occupied by background tasks such as `IOLoad`, while the inference task failed to make reliable forward progress. Watchdog reports also indicated that the idle task was unable to execute reliably. This is an important system-level symptom because

Table 6.4: Observed Starvation Behavior under Priority-C

Metric / Observation	**Priority-C Result**
Inference task priority	Lower than all background tasks
Background task behavior	Higher-priority sensing, communication, and I/O workloads
Inference period / deadline	10 ms / 10 ms
Target sample count	1000 inference activations
Experiment completion	Did not complete successfully
Valid timing statistics	Not available due to starvation
Observed failure symptom	Repeated watchdog triggers
Log evidence	Background tasks such as `IOLoad` frequently occupied processor time
System-level indication	Idle task unable to execute reliably
Interpretation	Unschedulable starvation configuration

FreeRTOS watchdog behavior can reveal cases where high-priority or continuously ready tasks prevent lower-priority and idle execution from occurring.

From a scheduling perspective, this behavior is consistent with starvation in a fixed-priority preemptive system. If higher-priority tasks become ready frequently enough, a lower-priority task may be delayed indefinitely or for a practically unacceptable amount of time. This type of behavior is especially relevant for embedded TinyML systems because inference is often added to a device that already performs sensing, communication, and storage. If inference is assigned too low a priority, it may not receive sufficient processor time to execute periodically.

Priority-C therefore identifies the practical lower bound of the evaluated scheduling configurations. Priority-A shows that inference can remain near-schedulable when protected by favorable priority assignment. Priority-B shows that increased contention can cause severe deadline degradation. Priority-C shows that extreme priority imbalance can prevent meaningful inference execution altogether. Together, the three configurations demonstrate a progression from acceptable timing behavior to degraded behavior and finally to starvation.

6.6 Memory Usage

In addition to timing behavior, memory usage was evaluated to determine whether the deployed TinyML system is feasible on the selected microcontroller-class platform. Memory is a critical constraint in TinyML deployments because inference frameworks, model parameters, input buffers, output tensors, and application tasks must coexist within limited RAM. TensorFlow Lite Micro is designed for such environments by using a statically allocated tensor arena rather than relying on dynamic allocation during inference [7, 8].

Table 6.5: Memory Usage of TinyML Inference System

Component	**Memory**
Tensor arena	40 KB
Heap usage (observed)	14.4 KB
Total RAM footprint	≈ 54 KB
Minimum free heap	279 KB

The memory results are summarized in Table 6.5. The TensorFlow Lite Micro model requires a tensor arena of 40 KB. Additional observed heap usage is approximately 14.4 KB, resulting in a total RAM footprint of about 54 KB for the inference-related components. Despite this allocation, the system maintains a minimum free heap of 279 KB.

These results indicate that memory capacity is not the limiting factor in the evaluated system. The model and inference runtime fit comfortably within the available RAM of the ESP32-S3 platform. The remaining heap headroom suggests that the observed timing failures are not caused by memory exhaustion. Instead, the results point to scheduling interference and processor-time contention as the dominant causes of deadline misses and starvation.

This distinction is important for embedded AI design. A model may satisfy memory constraints and still fail real-time constraints. Conversely, reducing model size alone may not eliminate deadline misses if the primary source of delay is scheduler interference from competing tasks. Therefore, optimization efforts should not focus only on model compression or memory reduction. They should also consider task priority, workload phasing, communication timing,

logging overhead, and I/O behavior.

6.7 Chapter Summary

This chapter presented the timing and memory results of the evaluated TinyML inference system under multiple FreeRTOS scheduling configurations. Priority-A showed near-schedulable behavior, with a mean latency below the 10 ms deadline and a deadline miss rate of 1.50%. Priority-B showed severe timing degradation, with mean latency increasing to 38363.46 μs and the deadline miss rate increasing to 59.00%. Priority-C demonstrated starvation behavior, where the inference task failed to make sufficient progress to complete the experiment.

The memory results showed that the deployed TinyML model fits within the resource constraints of the ESP32-S3 platform, requiring approximately 54 KB of RAM while leaving substantial free heap. Therefore, the primary limitation observed in this study is not memory capacity, but scheduling interference and processor-time contention.

Overall, the results demonstrate that real-time TinyML deployment requires more than selecting a small model. The inference workload must be integrated into the embedded system with careful attention to task priorities, background workload behavior, and worst-case timing effects. The next chapter discusses the implications of these findings, the limitations of the study, and possible directions for improving real-time TinyML schedulability.

Chapter 7

Discussion

This chapter discusses the implications of the experimental results presented in Chapter 6. While the previous chapter focused on reporting the measured timing and memory behavior of the evaluated scheduling configurations, this chapter interprets those results in relation to the broader research problem: whether TinyML inference can maintain real-time behavior when deployed as part of a multitasking embedded system.

The discussion is organized around five main themes. First, the chapter interprets the effect of scheduling configuration on inference latency, deadline misses, and starvation behavior. Second, it explains why model-level efficiency alone is insufficient for real-time TinyML deployment. Third, it presents scheduling-aware mitigation guidelines derived from the experimental results. Fourth, it discusses the energy implications of timing instability. Finally, it identifies the limitations of the study and explains which findings are likely to generalize beyond the specific platform and workload used in this thesis.

7.1 Interpretation of Scheduling Effects

The experimental results demonstrate that real-time TinyML inference on the ESP32-S3 is feasible under controlled scheduling conditions, but his ighly sensitive to scheduler-level interference. Under the Priority-A configuration, the inference task was assigned a higher priority than the background sensing, communication, and I/O tasks. In this configuration, the task achieved a mean latency below the 10 ms deadline and missed only 1.50% of deadlines. This result indicates that the deployed model is computationally feasible on the target platform when the scheduler gives inference sufficient execution preference.

However, Priority-A should not be interpreted as a strict hard real-time guarantee. Although the mean latency was below the deadline, the experiment still produced occasional long-tail latency

spikes, including a maximum observed latency of 23289 μs. These outliers show that favorable average-case timing does not fully eliminate timing risk. In real-time systems, a task can appear acceptable under average-case analysis while still violating deadlines due to sporadic interference, operating system overhead, interrupt activity, logging behavior, or other runtime effects. Therefore, the Priority-A result is best described as near-schedulable rather than strictly schedulable.

The Priority-B configuration shows a much stronger effect of scheduler-level interference. In this configuration, competing background tasks were assigned a higher priority than the inference task. As a result, the mean inference latency increased to 38363.46 μs, and the deadline miss rate increased from 1.50% to 59.00%. This marks a clear transition from near-schedulable to unschedulable behavior. The large standard deviation and maximum observed latency of 350920 μs show that the inference task did not merely become slower on average; it became highly unpredictable.

This behavior is consistent with established real-time scheduling theory. In fixed-priority preemptive systems, the response time of a task depends not only on its own execution time but also on interference from higher-priority tasks, possible blocking, and operating-system overheads [11, 12, 10]. In this thesis, the inference task itself is relatively lightweight and stable when considered in isolation. However, once it competes with higher-priority background workloads, its response time becomes dominated by scheduler-level interference rather than by the neural network computation alone.

The Priority-C configuration further highlights the practical limit of this behavior. Under Priority-C, the inference task was assigned lower priority than all major background workloads. The experiment did not complete successfully because the inference task was effectively starved. Repeated watchdog triggers indicated that the system could no longer provide a viable execution environment for periodic inference. This result is important because it shows that the failure mode is not limited to delayed inference results. Under sufficiently severe priority imbalance, the system may stop producing reliable inference results altogether.

Priority-C should therefore be interpreted as an experimentally observed starvation condition rather than as a missing quantitative dataset. The absence of complete timing statistics is itself

meaningful because the task could not make sufficient forward progress to generate those statistics. Prior formal work on FreeRTOS scheduling has shown that starvation and subtle multitasking failures can arise from complex task interactions and may be difficult to expose through ordinary testing alone [9]. The Priority-C experiment provides a practical demonstration of this issue in the context of TinyML inference.

Taken together, the three priority configurations show a progression of real-time behavior. Priority-A demonstrates that inference can remain near-schedulable when protected by favorable priority assignment. Priority-B shows that increased contention can cause severe latency growth and frequent deadline misses. Priority-C shows that extreme interference can produce starvation. This progression supports the central argument of this thesis: the feasibility of real-time TinyML depends not only on model efficiency but also on the scheduling environment in which the model executes.

7.2 Model Efficiency Is Necessary but Not Sufficient

A major implication of the results is that model efficiency alone is not sufficient to guarantee real-time TinyML behavior. TinyML research and deployment practice often emphasize model compression, quantization, tensor arena size, and kernel optimization. These concerns are essential because embedded platforms have limited memory, limited processor capacity, and strict energy constraints [8, 7, 1]. However, the results of this thesis show that a model that fits in memory and runs quickly in isolation can still fail when deployed in a multitasking real-time environment.

The memory results confirm this point. The deployed inference system required approximately 54 KB of RAM, including a 40 KB tensor arena and approximately 14.4 KB of observed heap usage. The system still maintained substantial free heap during execution. This indicates that memory capacity was not the limiting factor in the evaluated experiments. Instead, the dominant limitation was processor-time availability under different scheduling configurations.

This distinction is important because embedded developers may incorrectly assume that once a model fits on the microcontroller and has an acceptable isolated inference time, it is ready for

deployment. The Priority-B and Priority-C results contradict that assumption. The same model that was near-schedulable under Priority-A became unschedulable under Priority-B and starved under Priority-C. No model architecture changed between these configurations. The primary change was the scheduling relationship between inference and competing tasks.

Therefore, TinyML deployment should be treated as a system-level integration problem. The model must be efficient, but it must also be scheduled appropriately. Sensor acquisition, communication, storage, logging, and inference must be designed together. A real embedded device rarely performs inference alone. It must read sensors, exchange data, handle network activity, write logs, respond to interrupts, and maintain system services. Each of these activities can interfere with inference timing if not bounded and prioritized carefully.

The results also show why average latency is not enough. Under Priority-A, the mean latency was below the deadline, but deadline misses still occurred. Under Priority-B, some activations still completed quickly, but the overall distribution included a large long-latency tail. These observations show that real-time TinyML evaluation should include tail latency, observed worst-case latency, deadline miss ratio, and lateness, not only mean inference time. For time-sensitive embedded systems, the question is not simply whether the model can run, but whether it can run at the right time.

7.3 Scheduling-Aware Mitigation Guidelines

The experimental results suggest that improving TinyML timing behavior requires more than optimizing the neural network model itself. Since the same inference workload exhibits substantially different timing behavior under different priority assignments, mitigation should also be applied at the scheduling and system-design level. Based on the observed behavior across the evaluated configurations, this section derives several practical guidelines for real-time TinyML system design.

7.3.1 Measure Isolated Inference Latency Before Integration

The first guideline is to measure isolated inference latency before integrating the model into the full multitasking system. This establishes a baseline for the computational cost of the model and provides an estimate of the timing slack available before background workloads are introduced.

For example, if isolated inference already consumes most of the deadline interval, then even small amounts of scheduler interference may cause deadline misses. In contrast, if isolated inference consumes only a small portion of the deadline, the system has more tolerance for background activity. This baseline is necessary because it separates model-level cost from system-level interference.

However, isolated inference latency should not be treated as the final real-time result. It should be viewed only as the starting point for integration. The Priority-B and Priority-C results show that the actual response time of inference can become much larger than the isolated computation time when competing tasks interfere with execution.

7.3.2 Assign Inference Priority According to Timing Criticality

The second guideline is to assign inference priority according to the timing importance of the inference result. If inference is part of a periodic monitoring, alerting, or control loop, it should not be placed below long-running or high-duty-cycle background tasks. The Priority-B results show that assigning higher priority to competing background workloads can cause frequent deadline misses. The Priority-C results show that assigning inference below sustained high-priority workloads can cause starvation.

This does not mean that inference should always be the highest-priority task in every embedded system. Some safety-critical sensing or control tasks may legitimately require higher priority than inference. However, the priority assignment should reflect the functional role of the inference result. If inference output is required at a fixed rate, then the task must be given enough priority and processor time to satisfy that rate.

A practical design approach is to classify tasks according to timing criticality. Tasks that directly affect safety or have hard timing constraints should receive the strongest priority protection.

Periodic inference tasks used for real-time classification or alert generation should be placed above noncritical storage, logging, or best-effort communication tasks. Background activities should be assigned lower priority unless they have stricter timing requirements than inference.

7.3.3 Bound Background Workload Duration and Frequency

The third guideline is to bound background workloads in both duration and frequency. Communication, sensing, and I/O tasks should avoid long uninterrupted execution bursts. Instead, these activities should be divided into smaller execution segments where possible. This allows the scheduler to return control to the inference task more predictably.

This guideline is especially important for workloads such as cryptographic communication processing, file-system writes, or repeated sensor processing loops. Even when such tasks are individually reasonable, they can create substantial interference if they are executed too frequently or for too long. In fixed-priority scheduling, a lower-priority inference task cannot resume until higher-priority ready tasks yield, block, or complete. Therefore, long high-priority execution intervals can directly inflate inference response time.

Bounding background workloads can be implemented in several ways. Communication tasks can process a limited number of messages per activation. Storage tasks can buffer data and perform writes in smaller chunks. Sensor tasks can avoid unnecessary busy-waiting. I/O-heavy work can be moved to lower-priority tasks or executed less frequently. These design choices reduce the chance that inference will be delayed across multiple periods.

7.3.4 Keep Logging Outside the Inference Critical Path

The fourth guideline is to keep logging and file-system activity outside the inference-critical path whenever possible. In this thesis, SPIFFS-based logging was useful for persistent measurement collection. However, storage operations can introduce timing variability if they directly compete with inference execution. Flash-based file systems may have variable write latency due to buffering, wear leveling, garbage collection, or internal synchronization.

For experimental measurement, logging is necessary. For deployed systems, however, logging should be designed carefully so that it does not compromise real-time behavior. A practical approach is to record timing values into memory buffers during inference and defer file writes to a lower-priority logging task. This separates the timing-critical inference path from slower storage operations. If persistent logging is required, writes should be batched and scheduled during periods when inference is not expected to execute.

This approach also improves the quality of measurement. If logging occurs directly inside the measured critical path, the resulting latency values may include logging overhead rather than inference behavior alone. Separating measurement collection from persistent storage helps distinguish inference cost from instrumentation cost.

7.3.5 Treat Starvation as a Schedulability Failure

The fifth guideline is to treat starvation as a schedulability failure rather than ordinary performance degradation. Priority-C demonstrates that when inference is placed below sustained high-priority background work, it may fail to make sufficient progress. In such cases, reporting only average latency is impossible because the task may not complete enough activations to produce a valid latency distribution.

For practical embedded systems, starvation should be treated as a design failure. A system that occasionally produces late inference results may still be usable in some soft real-time applications. A system that stops producing inference results altogether is much more serious. Therefore, validation should include not only latency and deadline miss measurements but also evidence that the inference task continues to execute reliably over the full operating interval.

Watchdog triggers, failure to reach the expected number of inference activations, and repeated absence of inference logs should be treated as evidence of starvation or severe overload. These symptoms should lead to a redesign of task priorities, workload periods, or workload execution bounds.

7.4 Optional Mitigation-Oriented Priority Configuration

One possible extension of this work is to introduce an additional mitigation-oriented configuration, referred to here as Priority-D. Unlike Priority-A, Priority-B, and Priority-C, which are primarily used to characterize the effect of increasing contention, Priority-D would be designed to evaluate recovery after applying scheduling-aware mitigation.

The purpose of Priority-D would not be to add another stress case. Instead, it would test whether the system becomes schedulable again after correcting the priority imbalance observed in Priority-B and Priority-C. For example, inference priority could be protected or raised above high-duty-cycle background tasks, while background work remains present but is bounded more carefully. Communication and I/O operations could be divided into shorter segments, and logging could be deferred to a lower-priority task.

Such a configuration would strengthen the thesis by connecting the discussion directly to an experimentally testable mitigation strategy. Priority-C shows the failure mode under extreme interference. Priority-D would show whether that failure mode can be reduced or avoided through scheduling-aware design. This would help demonstrate not only that scheduler interference is harmful, but also that the harm can be mitigated through appropriate task organization.

Because the current conference paper is constrained by page limits, Priority-D may be more suitable for the thesis than for the shorter paper. In the thesis, it could be presented either as an additional experiment, if time allows, or as a proposed future extension. If implemented, Priority-D should be evaluated using the same metrics as Priority-A and Priority-B: mean latency, standard deviation, observed worst-case latency, deadline miss rate, and worst-case lateness. This would allow direct comparison between the baseline, degraded, starvation, and mitigation-oriented configurations.

7.5 Energy Implications

Although this thesis does not directly measure energy consumption, the findings remain relevant for energy-constrained embedded systems. TinyML systems are often motivated by the need to perform local inference on low-power devices. In such systems, energy efficiency depends not only on the neural network model but also on how predictably the processor can execute work and return to low-power states.

Increased scheduling interference can prolong active processing time. If inference is delayed by background tasks, the processor may remain active longer than expected. If deadline misses occur frequently, the system may spend additional time handling late work, logging errors, or recovering from overload. If watchdog triggers occur, the system may waste energy during repeated failure and recovery cycles. Therefore, timing instability can indirectly reduce energy efficiency even when the model itself is small.

Predictable timing also supports duty cycling. In a duty-cycled system, the processor wakes up, performs required work, and then returns to a low-power state. This strategy depends on predictable execution duration. If the inference task sometimes completes in approximately 8 ms but occasionally takes tens or hundreds of milliseconds due to scheduling interference, then duty-cycle planning becomes difficult. The system must either reserve energy for the worst case or risk failing to complete work before the next activation.

Prior work on IoT energy profiling emphasizes the importance of measuring and optimizing active time in embedded and IoT devices [13]. The results of this thesis suggest that scheduling design should be considered part of that optimization process. A scheduling configuration that reduces latency variability may improve both real-time correctness and energy predictability.

This does not imply that the lowest-latency configuration is always the most energy-efficient. Assigning inference very high priority may improve timing but could affect the execution pattern of other tasks. Similarly, deferring communication or storage may reduce inference interference but change network or file-system behavior. Direct energy measurement would be required to

quantify these tradeoffs. Nevertheless, the results indicate that timing predictability is an important precondition for energy-aware TinyML deployment.

7.6 Generalizability of Findings

The specific numerical results reported in this thesis are tied to the selected platform, model, task periods, workload implementation, and 10 ms deadline. Therefore, the exact mean latencies, deadline miss rates, and worst-case lateness values should not be treated as universal constants. A different microcontroller, neural network model, clock frequency, RTOS configuration, or workload mix would produce different numerical results.

However, the main trend is expected to generalize to other fixed-priority RTOS-based embedded systems. When a periodic inference task competes with higher-priority or high-duty-cycle background workloads, its response time can become dominated by scheduler-level interference. This is not specific to the ESP32-S3 or to the particular model used in this study. It follows from the general behavior of preemptive fixed-priority scheduling, where higher-priority ready tasks execute before lower-priority tasks.

The results are especially relevant to embedded systems that combine local inference with sensing, communication, and storage. These are common components of IoT and edge-intelligence applications. MQTT-based communication, cloud-connected sensing, and real-time alerting systems all introduce communication and processing demands that may interfere with local inference if not scheduled carefully [4, 5, 6, 3, 2]. Therefore, the scheduling concern identified in this thesis applies broadly to embedded AI systems that operate as part of a larger connected device.

The findings may also apply to multicore microcontrollers, but with important qualifications. Multicore execution can reduce interference by assigning inference and background workloads to separate cores. However, multicore systems may still experience interference through shared memory, caches, peripherals, file systems, communication stacks, interrupts, and synchronization mechanisms. Therefore, multicore execution may reduce but not eliminate the need for scheduling-aware design.

7.7 Limitations and Threats to Validity

This study has several limitations that should be considered when interpreting the results.

7.7.1 Single Hardware Platform

All experiments were conducted on a single embedded platform, the ESP32-S3. This allows the study to focus on controlled comparison across scheduling configurations, but it limits external validity. Different microcontrollers may have different processor speeds, memory hierarchies, interrupt behavior, RTOS ports, and peripheral performance. As a result, the exact timing results may differ on other platforms.

Future work should repeat the experiments on additional microcontroller platforms, including both single-core and multicore devices. This would help determine how much of the observed behavior is platform-specific and how much reflects general fixed-priority scheduling behavior.

7.7.2 Single Model and Application Type

The evaluation uses one deployed TinyML model and one application style based on periodic sensor-driven inference. This is appropriate for isolating the scheduling effects studied in this thesis, but it does not cover all possible TinyML workloads. Other models, such as keyword spotting, anomaly detection, image classification, or larger convolutional models, may have different execution times and memory requirements.

Future studies should evaluate multiple model types and input modalities. This would help determine whether certain model structures are more sensitive to scheduler interference than others. For example, models with longer inference times may have less slack, while shorter models may tolerate more background interference.

7.7.3 Controlled Background Workloads

The background tasks used in this study are controlled representations of sensing, communication, and I/O activity. They are designed to emulate common embedded workload patterns while allowing repeatable experimentation. However, real deployments may include more complex network stacks, sensor drivers, interrupt patterns, wireless retransmissions, flash-storage delays, or application-specific processing.

Therefore, the workload should be interpreted as representative rather than exhaustive. The results show that scheduler interference can strongly affect inference timing, but they do not capture every possible source of interference in deployed systems. Future work should evaluate the system with real sensor devices, live wireless communication, and more diverse file-system behavior.

7.7.4 Observed WCET Rather Than Formal WCET

This study reports the maximum observed latency as the observed WCET. This is useful for empirical evaluation, but it is not equivalent to formally derived worst-case execution time. A longer experiment or different workload phasing could reveal a larger maximum latency. Therefore, observed WCET should be interpreted as the worst case seen during the experiment, not as a mathematical upper bound.

Formal WCET analysis is difficult for modern embedded systems because timing can be affected by interrupts, operating-system behavior, memory effects, peripheral activity, and workload interleavings. Nevertheless, combining empirical measurement with more formal response-time analysis would strengthen future work.

7.7.5 Fixed Deadline Configuration

The experiments use a fixed 10 ms period and deadline. This deadline was selected to match the periodic activation interval of the inference task and to represent a practical time-sensitive embedded sensing requirement. Keeping the deadline fixed helps isolate the effect of scheduling configuration. However, different applications may require different periods and deadlines.

Future work should evaluate multiple deadline settings. A relaxed deadline may make Priority-B less severe, while a tighter deadline may cause Priority-A to miss more deadlines. Varying the deadline would help identify the boundary between schedulable and unschedulable operations more precisely.

7.7.6 No Direct Energy Measurement

The study discusses energy implications but does not directly measure power or energy consumption. Therefore, claims about energy are interpretive rather than quantitative. The results suggest that timing instability may reduce duty-cycling predictability and prolong active processing time, but this should be validated with direct power measurement in future work.

7.8 Implications for Real-Time TinyML Design

The results of this thesis suggest several broader implications for real-time TinyML design.

First, TinyML inference should be treated as a real-time task when its output is time-sensitive. It should not automatically be treated as best-effort background computation simply because it is an AI workload. If the system depends on inference results for monitoring, alerting, or control, then inference timing must be included in the system design.

Second, model benchmarking should be performed under realistic multitasking conditions. A model that meets a deadline in isolation may fail when integrated with communication, sensing, storage, and logging tasks. Therefore, isolated inference benchmarking should be followed by system-level scheduling evaluation.

Third, task priority assignment should be documented and justified. In many embedded systems, task priorities are selected informally during implementation. The results of this thesis show that priority assignment can determine whether inference is near-schedulable, unschedulable, or starved. Therefore, priority tables should be included as part of experimental reporting and system documentation.

Fourth, deadline miss rate and tail latency should be treated as first-class evaluation metrics.

Mean latency is useful, but it does not capture the worst behavior of the system. For real-time TinyML systems, deadline misses, lateness, and observed worst-case latency provide more meaningful insight into deployment readiness.

Finally, scheduling-aware mitigation should be considered alongside model optimization. Reducing model size and improving kernel performance are valuable, but they may not solve deadline misses caused by interference. A complete solution may require both model-level optimization and system-level scheduling control.

7.9 Chapter Summary

This chapter interpreted the experimental findings and discussed their implications for real-time TinyML deployment. The results show that the evaluated TinyML model is feasible on the ESP32-S3 from a memory and nominal execution-time perspective, but its real-time behavior depends strongly on scheduling configuration. Priority-A produced near-schedulable behavior, Priority-B produced severe deadline degradation, and Priority-C produced starvation.

The chapter argued that model efficiency is necessary but not sufficient for real-time TinyML. The system must also be designed to control scheduler interference, bound background workloads, protect time-critical inference, and avoid placing inference below sustained high-priority tasks. The discussion also identified energy implications, generalizability considerations, and study limitations.

Overall, the results reinforce the main conclusion of this thesis: real-time TinyML deployment must be evaluated as a complete embedded systems problem. Neural network efficiency, RTOS scheduling, workload composition, memory usage, logging behavior, and timing validation must be considered together to determine whether an embedded inference system can satisfy its real-time requirements.

Chapter 8

Conclusion

This thesis investigated the real-time behavior of TinyML inference on a microcontroller-class embedded platform under FreeRTOS scheduling. The study was motivated by the observation that TinyML research often emphasizes model compression, memory footprint, and isolated inference latency, while practical embedded deployments must execute inference alongside sensing, communication, storage, and other system-level tasks. As a result, a model that appears suitable in isolation may still fail to satisfy real-time requirements once integrated into a multitasking runtime environment.

The experimental results show that TinyML inference on the ESP32-S3 is memory-feasible and nominally fast. The deployed model required approximately 54 KB of RAM, including the TensorFlow Lite Micro tensor arena and observed heap usage, while leaving substantial free heap available during execution. Under favorable scheduling conditions, the inference task achieved a mean latency below the 10 ms deadline, demonstrating that the workload can execute within the required timing window when protected from excessive interference.

However, the results also show that memory feasibility and acceptable average latency are not sufficient to guarantee real-time correctness. Across the evaluated priority configurations, the same inference workload exhibited substantially different timing behavior. Under Priority-A, the system operated in a near-schedulable region, with a low deadline miss rate. Under Priority-B, increased background-task interference caused a sharp rise in mean latency, timing variability, and deadline misses, pushing the system into an unschedulable regime. Under Priority-C, the inference task was effectively starved and could not make sufficient progress to complete the experiment. These results demonstrate a clear transition from near-schedulable operation to severe deadline degradation and, under extreme interference, to starvation.

The main conclusion of this thesis is that real-time TinyML deployment must be treated as a system-level scheduling problem, not only as a model-optimization problem. Model compression,

quantization, and efficient inference kernels remain important, but they do not fully determine whether inference can meet deadlines in a multitasking embedded system. Task priorities, background workload duration, I/O behavior, communication activity, logging overhead, and scheduler interference all influence whether inference results are produced on time. Therefore, embedded AI systems require tighter integration between machine learning design and real-time systems engineering.

This work also highlights the importance of evaluating timing behavior using metrics beyond average latency. Deadline miss ratio, observed worst-case latency, lateness, and starvation behavior provide a more complete picture of real-time suitability. In particular, long-tail latency and watchdog-triggered starvation show that a system may appear efficient under nominal conditions while still failing under realistic runtime contention. For TinyML systems used in monitoring, alerting, or control applications, these timing failures may reduce system reliability even when the model itself is small and accurate.

Future work should focus on improving timing predictability through stronger temporal isolation between inference and background activities. One direction is to bound the execution duration of sensing, communication, and I/O tasks so that they cannot delay inference across multiple periods. Another direction is to move logging and file-system operations outside the inference-critical path, using buffering or deferred low-priority writes to reduce interference. Additional scheduling strategies beyond basic fixed-priority execution should also be explored, including time-triggered execution, reservation-based scheduling, and mixed-criticality task organization.

Further experiments are also needed to evaluate the robustness of the findings across broader conditions. Longer experiment durations would provide stronger evidence about rare timing outliers and observed worst-case behavior. Alternative model architectures, including different TinyML workloads such as keyword spotting, anomaly detection, and image-based inference, would help determine how model structure affects schedulability. Finally, multicore deployment strategies should be investigated to determine whether assigning inference and background workloads to separate cores can reduce interference and whether shared resources such as memory, peripherals,

communication stacks, and file systems still introduce timing variability.

Overall, this thesis demonstrates that TinyML inference can be deployed on microcontroller-class hardware, but real-time correctness depends on more than fitting the model into memory or achieving low isolated inference latency. Reliable embedded AI deployment requires coordinated design across the model, runtime, scheduler, and surrounding system workload. By experimentally characterizing the effect of scheduling interference on TinyML inference timing, this work contributes toward more predictable and dependable real-time embedded AI systems.

References Cited

[1] Z. Zhang and J. Li, "A review of artificial intelligence in embedded systems," *Micromachines*, vol. 14, no. 5, p. 897, 2023.

[2] A. S. M. Mohsin and M. A. Muyeed, "Iot based smart emergency response system (sers) for monitoring vehicle, home and health status," *Discover Internet of Things*, vol. 4, p. 22, 2024.

[3] H. Zhang, R. Zhang, and J. Sun, "Developing real-time iot-based public safety alert and emergency response systems," *Scientific Reports*, vol. 15, p. 29056, 2025.

[4] D. Borsatti, W. Cerroni, F. Tonini, and C. Raffaelli, "From iot to cloud: Applications and performance of the mqtt protocol," in *Proceedings of the 22nd International Conference on Transparent Optical Networks (ICTON)*, 2020, pp. 1–4.

[5] B. Bendele and D. Akopian, "A study of iot mqtt control packet behavior and its effect on communication delays," in *Proceedings of the IS&T International Symposium on Electronic Imaging, Mobile Devices and Multimedia: Enabling Technologies, Algorithms, and Applications*, 2017, pp. 120–1–120–8.

[6] N. Saha, P. Paul, K. Ji, and R. Harika, "Performance evaluation framework of mqtt client libraries for iot applications in manufacturing," *Manufacturing Letters*, vol. 41, pp. 1237–1245, 2024.

[7] R. David, J. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, S. Regev, R. Rhodes, T. Wang, and P. Warden, "Tensorflow lite micro: Embedded machine learning on tinyml systems," in *Proceedings of the 4th MLSys Conference*, 2021.

[8] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. Sebastopol, CA, USA: O'Reilly Media, 2020.

[9] C.-K. Lin and B.-Y. Wang, "Analyzing freertos scheduling behaviors with the spin model checker," *arXiv preprint arXiv:2205.07480*, 2022.

[10] B. D. Miranda, R. S. de Oliveira, and A. Carminati, "Analysis of freertos overheads on periodic tasks," in *Proceedings of the Brazilian Symposium on Computing Systems Engineering (SBESC)*, 2020.

[11] L. Sha, R. Rajkumar, and J. P. Lehoczky, "Priority inheritance protocols: An approach to real-time synchronization," *IEEE Transactions on Computers*, vol. 39, no. 9, pp. 1175–1185, Sep. 1990.

[12] S. Davari and L. Sha, "Sources of unbounded priority inversions in real-time systems and a comparative study of possible solutions," in *Proceedings of the ACM SIGPLAN Workshop on Language, Compiler, and Tool Support for Real-Time Systems*, 1992, pp. 110–118.

[13] I. Amirtharaj, "Energy measurement and profiling of internet of things devices," Master's thesis, Santa Clara University, Santa Clara, CA, USA, 2018.

[14] I.-C. Chao, S.-Y. Lin, K. B. Lee, F. Proctor, C.-C. Shen, and F.-R. Chang, "Precise latency measurement of unidirectional-data-flow network equipment," in *Proceedings of the IEEE International Symposium on Precision Clock Synchronization for Measurement, Control and Communication (ISPCS)*, 2012.

[15] J. L. Reyes-Ortiz, D. Anguita, A. Ghio, L. Oneto, and X. Parra, "Human activity recognition using smartphones," 2013, [Dataset], UCI Machine Learning Repository. doi: 10.24432/C54S4K.

Chapter A

Important Stuff

Include raw data, large tables, or long derivations here. [cite: 141]